



WiFi Access

SSID: **LE Guest**

Password: **B3OurGu3st**



Workshop Resources

Cursor Workshop

Learning Goals

Understand concepts and terminology
related to **AI assisted development**

Proficiency in using Cursor for **greenfield**
and **brownfield** development

Apply appropriate solutions for **enhancing**
Cursor's context in your project in ways
that are compliant with company practices

Kickoff



Workshop Resources

Setting the stage with our goals, agenda, and constraints

Introductions



Matt Eland



Tommy Allsup



Ben Grossman

Who We Are

At Leading EDJE, we believe technology is the most powerful tool for transforming businesses and unlocking human potential.

As strategists, innovators, and partners, we don't just solve problems — we craft bespoke solutions that positively disrupt your business.





Housekeeping Notes



WiFi Access

SSID: **LE Guest**
Password: **B3OurGu3st**



Slides & Materials

Available at
[LeadingEDJE/CursorWorkshop](#)



Stuck? Confused? Curious?

There's 3 of us here for a
reason

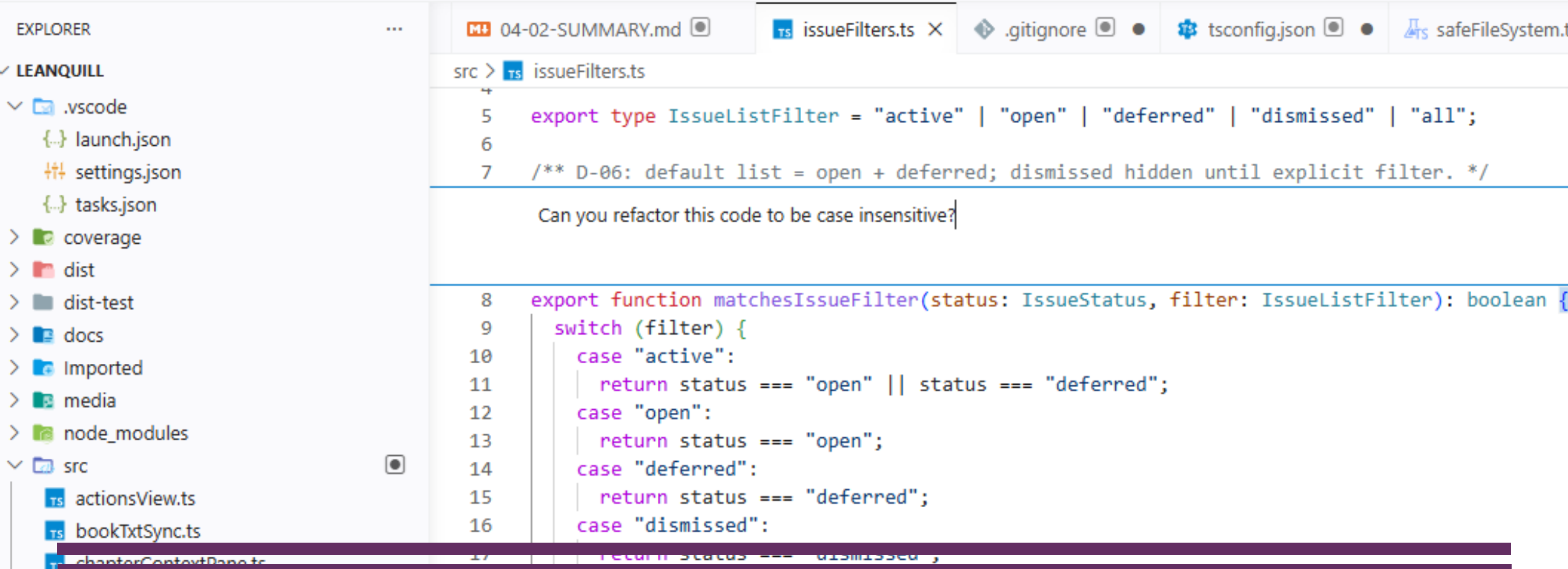


Bathrooms

Outside of our space in
the hall area
Floor doors are locked
and require a keyfob

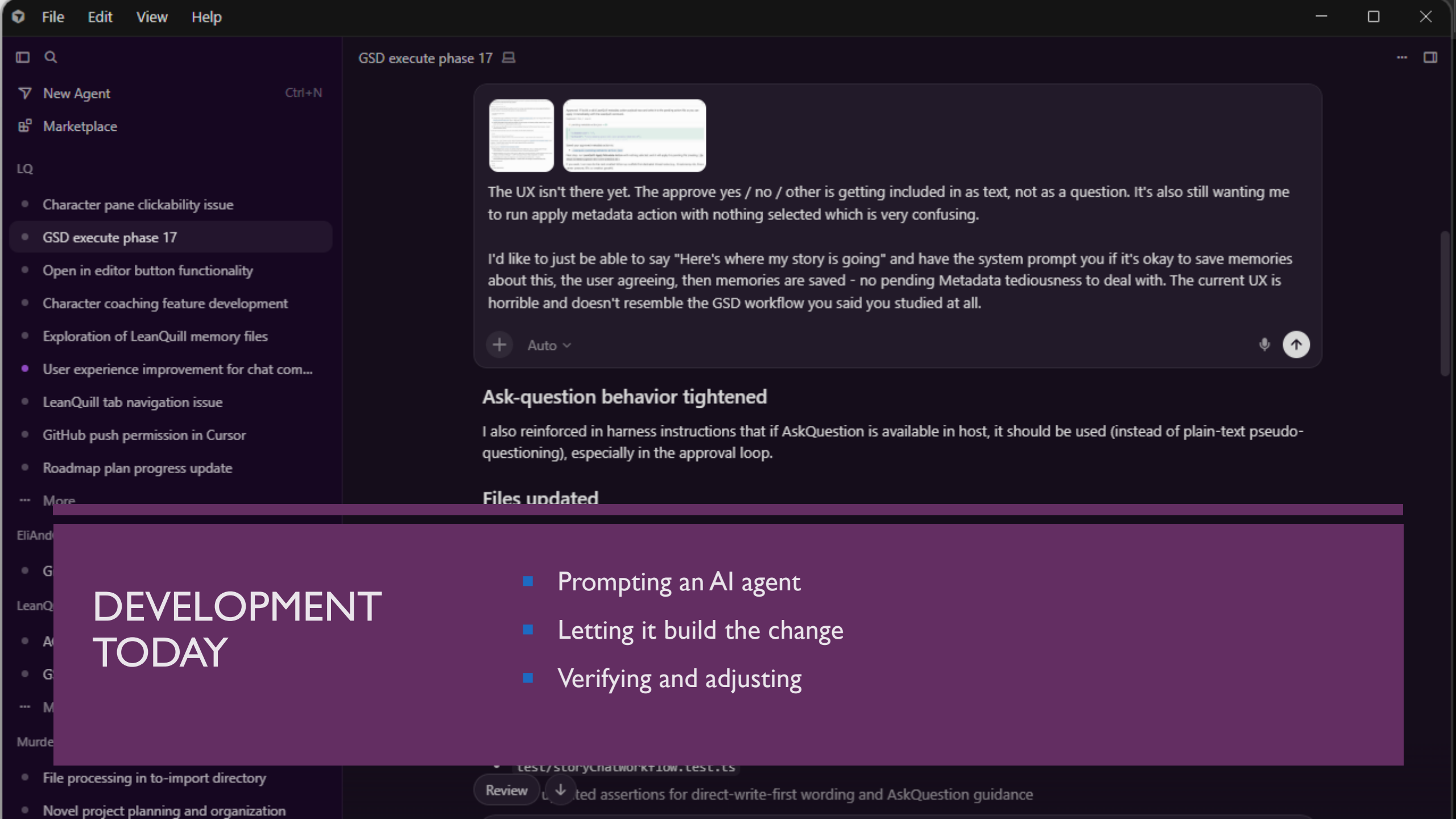
Rough Agenda

Time	Content
8 – 9:15 AM	Introductions, Terminology, and Intro to Cursor
9:30 – 10:30 AM	Tooling Discussion and Demonstration
10:45 – 11:45 AM	Cursor for Greenfield Development
11:45 AM – 12:45 PM	Lunch Break
12:45 – 1:30 PM	Cursor and Context Discussion
1:45 – 2:30 PM	Cursor for Brownfield Development
2:45 – 3:45 PM	Advanced AI Integration
4 – 5 PM	Final discussions and topics of interest



DEVELOPMENT 1 YEAR AGO

- Working in an IDE
- Handing off tricky or trivial tasks to AI agents
- Verifying / modifying results



New Agent Ctrl+N

Marketplace

LQ

- Character pane clickability issue
- **GSD execute phase 17**
- Open in editor button functionality
- Character coaching feature development
- Exploration of LeanQuill memory files
- User experience improvement for chat com...
- LeanQuill tab navigation issue
- GitHub push permission in Cursor
- Roadmap plan progress update

More

EliAnd

• G

LeanQ

• A

• G

• M

Murde

• File processing in to-import directory

• Novel project planning and organization

GSD execute phase 17



The UX isn't there yet. The approve yes / no / other is getting included in as text, not as a question. It's also still wanting me to run apply metadata action with nothing selected which is very confusing.

I'd like to just be able to say "Here's where my story is going" and have the system prompt you if it's okay to save memories about this, the user agreeing, then memories are saved - no pending Metadata tediousness to deal with. The current UX is horrible and doesn't resemble the GSD workflow you said you studied at all.

+ Auto



Ask-question behavior tightened

I also reinforced in harness instructions that if AskQuestion is available in host, it should be used (instead of plain-text pseudo-questioning), especially in the approval loop.

Files updated

DEVELOPMENT TODAY

- Prompting an AI agent
- Letting it build the change
- Verifying and adjusting

Review



ted assertions for direct-write-first wording and AskQuestion guidance



VIBE CODING

"Fully give in to the vibes, embrace exponentials, and forget that the code even exists."

- Andrej Karpathy – February 2025



AGENTIC ENGINEERING

"If an LLM wrote every line of your code, but you've reviewed, tested, and understood it all, that's not vibe coding - that's using an LLM as a typing assistant."

- Simon Willson

O'REILLY®



Beyond Vibe Coding

From Coder to AI-Era Developer



Addy Osmani

AI-ASSISTED ENGINEERING

"AI-assisted engineering is a more structured approach that combines the creativity of vibe coding with the rigor of traditional engineering practices. It involves specs, rigor and emphasizes collaboration between human developers and AI tools, ensuring that the final product is not only functional but also maintainable and secure."

- Addy Osmani (Google)

Full SDLC Integration:

- Specs
- Testing
- Review Gates
- Code Scanning

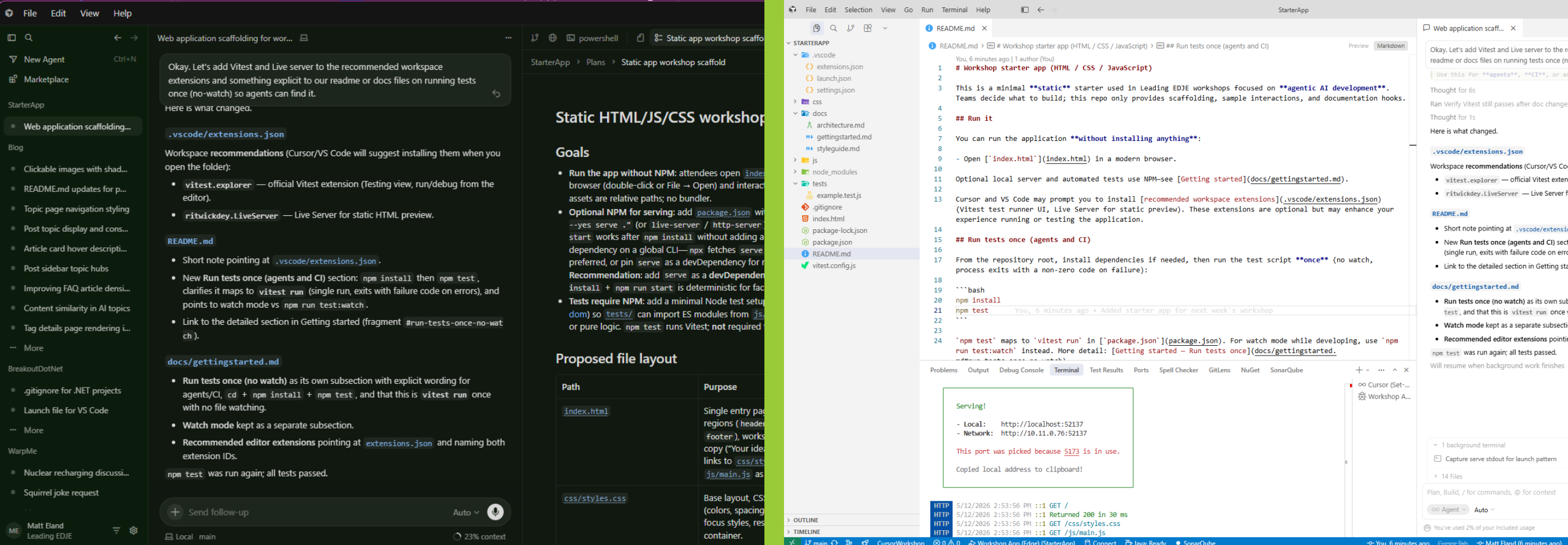
+

•

○

Cursor Basics

Getting oriented in the Cursor IDE



Cursor UIs

Sidebar Chat

(Ctrl+L)

The screenshot shows the Visual Studio Code interface with a C# file named `Program.cs` open. The file contains the following code:

```
1 using FastEndpoints;
2 using FastEndpoints.Swagger;
3 using MattEland.Jaimes.Repositories;
4 using MattEland.Jaimes.ServiceDefaults;
5 using System.Diagnostics;
6 using MattEland.Jaimes.ApiService.Helpers;
7 using MattEland.Jaimes.ServiceDefinitions.Services;
8 using MattEland.Jaimes.Services;
9
10 namespace MattEland.Jaimes.ApiService;
11
12 3 references | Matt Eland, last week | 3 authors (Matt Eland and others)
13 public class Program
14 {
15     0 references
16     public static async Task Main(string[] args)
17     {
18         WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
19
20         // Add service defaults & Aspire client integrations.
21         builder.AddServiceDefaults();
22
23         // Add services to the container.
24         builder.Services.AddProblemDetails();
25
26         // Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
27         builder.Services.AddOpenApi();
28
29         // Add Swagger services
30         builder.Services.AddEndpointsApiExplorer();
31
32         builder.Services.AddFastEndpoints().SwaggerDocument();
33
34         // Register a shared ActivitySource instance with the same name used by OpenTelemetry
35         builder.Services.AddSingleton(new ActivitySource(builder.Environment.ApplicationName ?? "Jaimes.ApiService"));
36
37         // Configure ChatOptions from configuration and register instance for DI
38         ChatOptions chatOptions = builder.Configuration.GetSection(key: "ChatService").Get<ChatOptions>() ?? throw new Invalid
39         builder.Services.AddSingleton(chatOptions);
40
41         // Add Jaimes repositories and services
42         builder.Services.AddJaimesRepositories(builder.Configuration);
```

The sidebar chat window is open, showing a message from the AI agent: "Refactor this code to reduce the Cyclomatic Complexity and improve the readability." The chat window also shows a "New Chat" button and a "Local" tab. The status bar at the bottom indicates the current file is `Program.cs` and the project is `JAIMES-AF`.

gram.cs 1

ListGamesEndpoint.cs

ListRulesetsEndpoint.cs

@ Games.razor

@ NewGame.razor M x

plan.plan.md

add-c

JAIMES AF.Web > Components > Pages > @ NewGame.razor

You, 1 second ago | 1 author (You)

1 @page "/games/new" You, 4 minutes ago • Enables game deletion ...

2

3 @rendermode InteractiveServer

4

5 <PageTitle>Create New Game</PageTitle>

6

<MudPaper Class="pa-4">

<MudText Typo="Typo.h4">Create New Game</MudText>

7 <MudContainer MaxWidth="MaxWidth.Medium" Class="mt-4">

8 <MudPaper Class="pa-4">

9 <MudText Typo="Typo.h4" Class="mb-4">Create New Game</MudText>

10

11 @if (isLoading)

12 {

13 <MudProgressCircular Indeterminate="true" Class="my-4" />

14 }

15 else if (!string.IsNullOrEmpty(errorMessage))

16 {

17 <MudAlert Severity="Severity.Error" Class="mb-4">@errorMessage</MudAlert>

18 }

19

20 @if (!isLoading)

21 {

22 <MudForm>

23 <MudSelect @bind-Value="selectedScenarioId"

24 Label="Scenario"

25 Variant="Variant.Outlined"

26 Class="mb-4"

27 Disabled="isCreating">

28 @foreach (var scenario in scenarios)

29 {

Implement new game page functionality

Implement the New Game page in such a way that it allows the user to specify the details necessary to create a new game. Once the user clicks a Create button the page should call to the NewGameEndpoint.cs and then navigate to GameDetails.razor to display the details of the new game once created.

Agent Auto

NewGame.razor +59 -4

@using MattEland.Jaimes.ServiceDefinitions.Responses

@rendermode InteractiveServer

<PageTitle>Create New Game</PageTitle>

Checking for linting errors:

Found 1 error in 1 file

Removing the unnecessary using directive:

NewGame.razor -1

@using MattEland.Jaimes.ServiceDefinitions.Responses

@rendermode InteractiveServer

No linter errors found

Checking how buttons are used in other components to ensure consistency:

Grepped MudButton.*OnClick in JAIMES AF.Web

Updating the button to match the pattern used in other components:

NewGame.razor +2 -2

<MudProgressCircular Class="ms-n1" Size="Size.Small" Indeterminate="true" />

Chat: Agent

Cursor Allowlist

M

matt.eland@gmail.com

Pro Plan

Search settings Ctrl+F

⚙️ General

∞ Agents

→ Tab

📦 Models

☁️ Cloud Agents

🔗 Tools & MCP

📋 Rules, Memories, Commands

📖 Indexing & Docs

🌐 Network

>> Beta

📖 Docs

🔖

Auto-Run

Auto-Run Mode

Choose how Agent runs tools like command execution, MCP, and file writes.

Use Allowlist ↕

Command Allowlist

Commands that can run automatically

npm test X cd X mvn X

MCP Allowlist

MCP tools that can run automatically. Format: 'server:tool', 'server:*' for all tools from a server, '*:tool' for a tool from any server, or '*:*' for all tools from all servers

e.g., filesystem:read_file

Browser Protection

Prevent Agent from automatically running Browser tools



File-Deletion Protection

Prevent Agent from deleting files automatically



Dotfile Protection

Prevent Agent from modifying dot files like .gitignore automatically



External-File Protection

Prevent Agent from creating or modifying files outside of the workspace automatically



Add Delete Games and New Game Link

Backend Changes

1. Service Layer

- File: JAIMES AF.ServiceDefinitions/Services/IGameService.cs
- Add method: Task DeleteGameAsync(Guid gameId, CancellationToken cancellationToken = default)
- File: JAIMES AF.Services/Services/GameService.cs
- Implement DeleteGameAsync method
- Find game by ID, validate it exists
- Delete game (Messages will cascade delete automatically per EF Core configuration)

2. API Endpoint

- New File: JAIMES AF.ApiService.Endpoints/DeleteGameEndpoint.cs
- FastEndpoints endpoint using Inheritance pattern
- Route: DELETE /games/{gameId:guid}
- Call GameService.DeleteGameAsync
- Return 204 No Content on success, 404 if game not found

3. Testing

- New File: JAIMES AF.Tests/Endpoints/DeleteGameEndpointTests.cs
- Test successful deletion
- Test 404 when game doesn't exist
- Follow pattern from ListGamesEndpointTests.cs

Frontend Changes

1. Games List Page

- File: JAIMES AF.Web/Components/Pages/Games.razor
- Add "New Game" button/link at top (always visible, before the table)

Determine a scope of changes for adding the ability to delete existing games and a link to a new games page.

This new game link should be present in all cases and should also be linked to from the (no games) message. It should navigate the browser to /games/new and should not immediately create a game or call the API.

The delete game link should occur at the right edge of each row, use an appropriate icon and color, and require the user to confirm before deletion. Actual deletion should occur via the API using a DELETE verb and a new endpoint.

Plan Auto @ 🌐 🖼️ ⬆️

Searched What MudBlazor components are used for icons and buttons in the web UI? (J...
Read GameDetails.razor
Grep'd MudDialog|Dialog|Confirm in JAIMES AF.Web
Read JaimesDbContext.cs

Checking the Message entity structure and reviewing an endpoint test to understand the testing pattern:

Explored 1 directory 2 files
Creating the plan:

add-delete-games-and-new-game-link.plan.md

Add Delete Games and New Game Link

Add ability to delete games with confirmation dialog, and add a "New Game" link that navigates to /games/new without creating a game. The delete button appears at the right edge of each game row, and the new game link is always visible including in the empty state.

- 6 To-dos
- ☐ Add DeleteGameAsync method to IGameService interface and implement in GameService
 - ☐ Create DeleteGameEndpoint with DELETE /games/{gameId:guid} route
 - ☐ Add tests for DeleteGameEndpoint
 - ... 3 more

View Plan Auto Build Ctrl+d

Chat: Plan

Fixing incorrect EF migrations

+🕒⋮

I think the ApiService has an incorrect set of migrations. When the application starts I get this exception: "There is already an object named 'Rulesets' in the database."

I think this comes from a bad series of EF migrations. What should I do to resolve this?

🗨 Ask ▾

Auto ▾

@🌐📷⬆

Investigating the EF migrations issue. Checking migrations and how they're applied.

Explored 1 directory 3 files

Checking the migrations for duplicate table creation:

Explored 1 file 2 searches

Checking other migrations for duplicate table creation:

Explored 2 files 1 search

Checking the migration history and the model snapshot:

Explored 2 files >

The error "There is already an object named 'Rulesets' in the database" usually means the `Rulesets` table exists, but the `__EFMigrationsHistory` table is missing or incomplete, so EF tries to re-run `InitialCreate`.

Diagnosis

Your migrations look correct:

- `20251029024204_InitialCreate.cs` creates the `Rulesets` table
- Later migrations only modify data or add columns; none recreate the table

The issue is likely a mismatch between the database state and the migration history.

Chat: Ask

Enhance game visuals for reversi

[Create PR](#)[Review](#)

28m · +381 -67 · Auto



Okay, this is awesome, but I have a couple of requests. First, can you make the preview dots the same size as the pieces?

Secondly, the orange text when the AI is thinking is hard to read on a white background.



- Added hover tracking with `mouseenter` and `mouseleave` listeners
- Modified `BoardCell` to support a highlighted state for capture preview
- Updated piece rendering to show purple gradient when highlighted

All linting errors are resolved, and the code is ready to use. The game now provides clearer visual feedback for valid moves and potential captures.



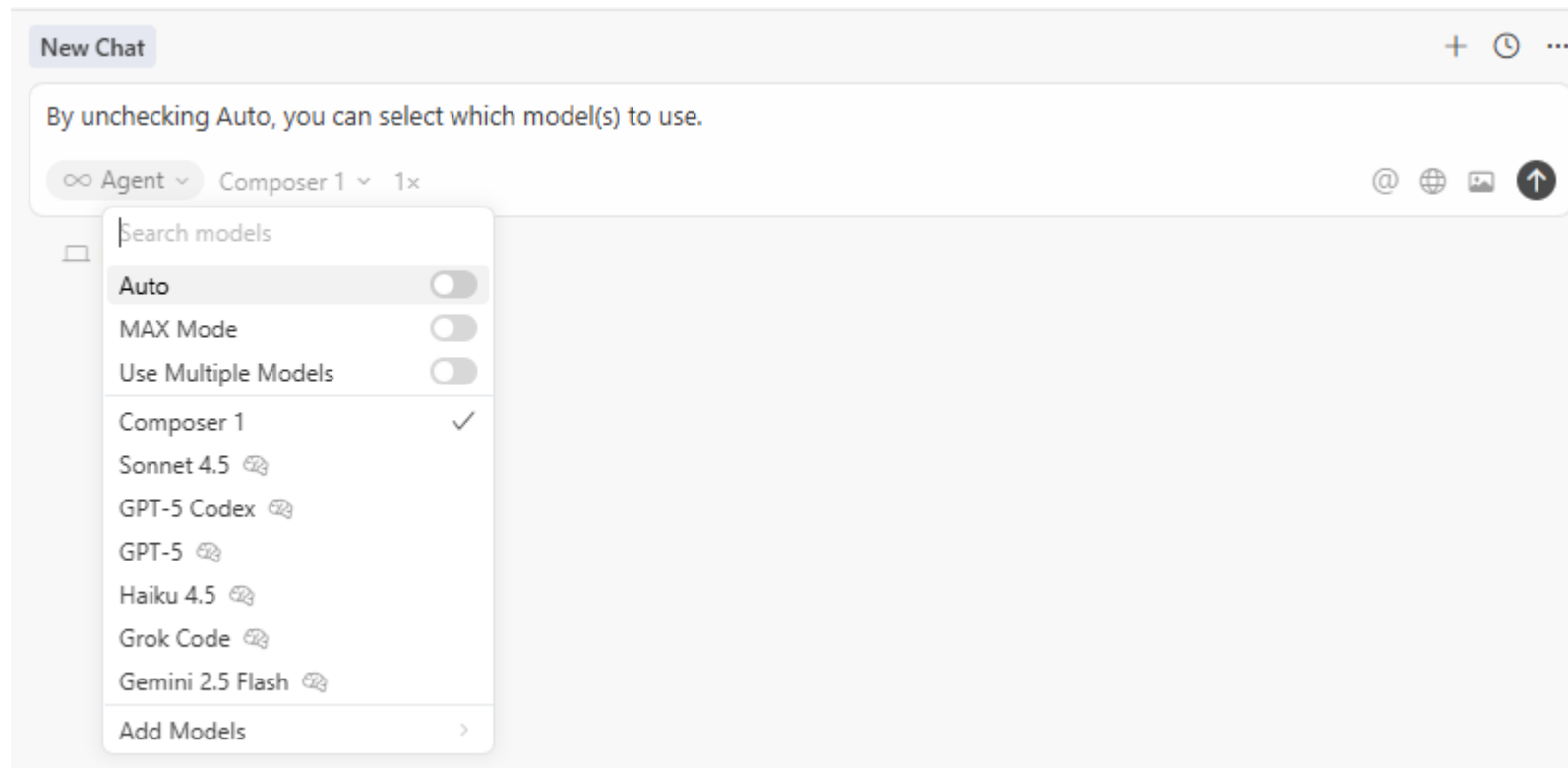
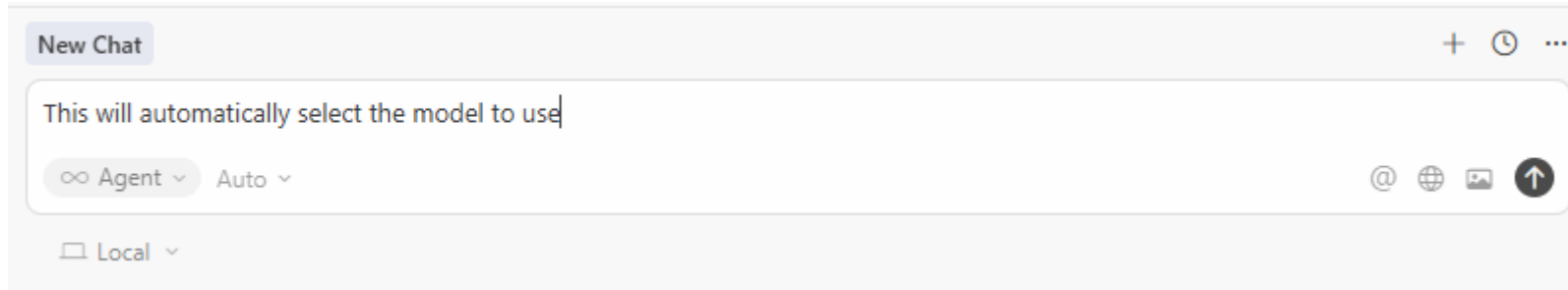
This is great, but can we make the potential move ones not have a gradient, just have a large circle?

∞ Agent ▾ Auto ▾



Chat with Visuals

Model Selection



Model Selection

M matt.eland@gmail.com
Free Plan

Search settings Ctrl+F

- General
- Agents
- Tab
- Models**
- Cloud Agents
- Tools & MCP
- Rules, Memories, Commands
- Indexing & Docs
- Network
- Beta
- Docs

Models

Add or search model

Composer 1	☒
Sonnet 4.5	☒
GPT-5 Codex	☒
GPT-5	☒
Haiku 4.5	☒
Grok Code	☒
Gemini 2.5 Flash	☒
Sonnet 4.5	☐
GPT-5 Codex High	☐
GPT-5 Codex Fast	☐

[View All Models](#)

API Keys

OpenAI API Key ☐

You can put in [your OpenAI key](#) to use OpenAI models at cost.

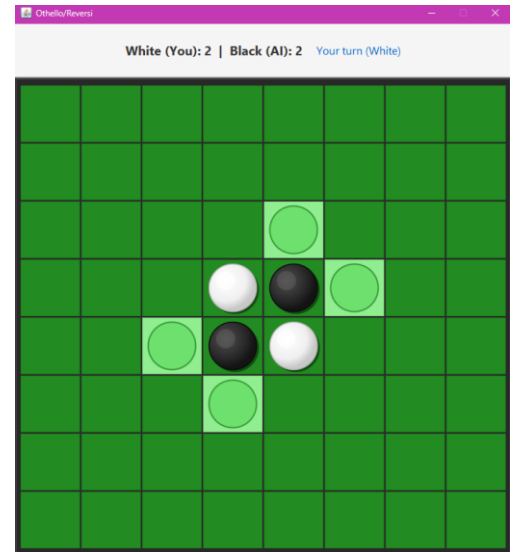
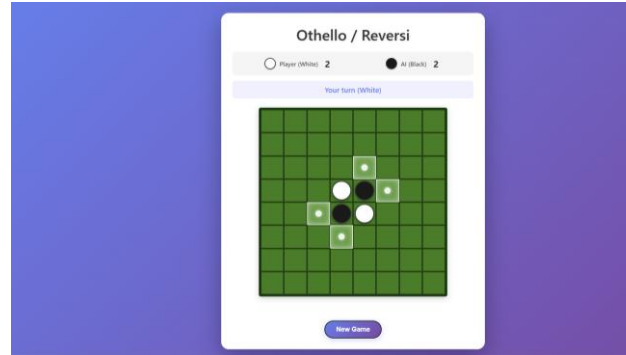
Enter your OpenAI API Key

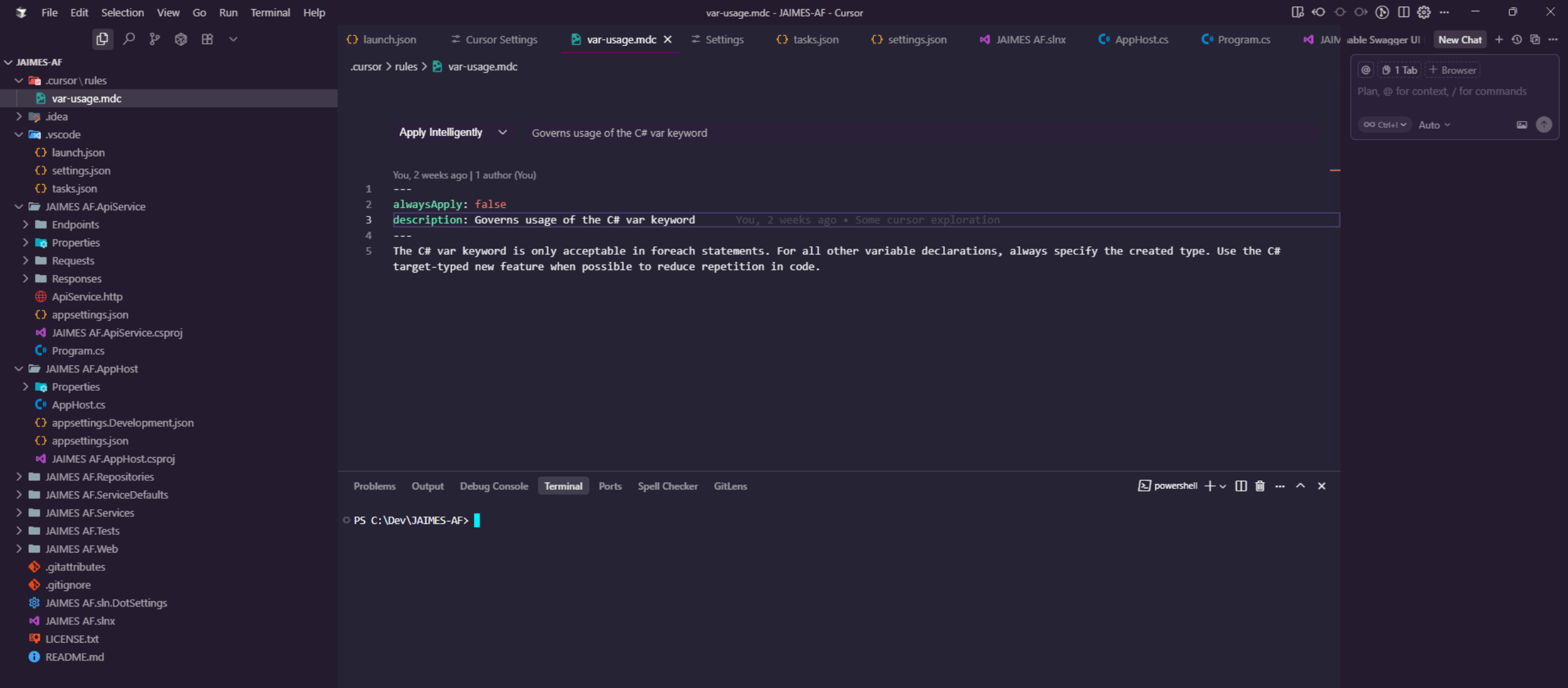
Override OpenAI Base URL ☐

Change the base URL for OpenAI API requests.

Demo

Building a Greenfield
Application with Cursor





Cursor

Cursor vs Copilot

CURSOR

- Separate IDE (forked from VS Code)
- Can run in cloud if with per-use billing
- Heavy focus on context
- Has Enterprise editions
- Support for MCP, Agent, Plan, and Ask
- Supports many models
- Generally feels faster

COPILOT

- VS Code Extension
- GitHub Cloud Runners
- Heavy focus on GitHub Integration
- Focus on Enterprise-level Features
- Support for MCP, Agents, and Ask
- Supports many models

Some Agentic AI IDEs

VS Code w.
Copilot

Claude
Code

Antigravity

Windsurf
2.0

Cursor
3.0

Zed Editor

TabbyML

Void

Tabnine

Terminology

Laying a foundation for the rest of today's discussions and demos

Improve loading display layout

[Create PR](#)[Review](#)

Now · +46 -39 · Auto

At present the loading display in `Games.razor` causes the existing layout to flow around the spinner and looks unprofessional. I'd like a more consistent layout during the loading experience that doesn't cause the text on the page to move around when the spinner goes away.

Explored 2 files 1 search

The `if-else` chain causes layout shifts. Reserving space for the content area and overlaying the spinner to prevent movement. Updating the layout:

`Games.razor` +46 -39

```
<div style="position: relative; min-height: 200px;">
  @if (isLoading)
  {
    <div style="display: flex; justify-content: center; align-items: center; min-height: 200px;">
      <MudProgressCircular Indeterminate="true"/>
    </div>
  }
```

Checking for linting errors:

No linter errors found in `Games.razor`

Updated the loading display to prevent layout shifts.

Changes made:

1. **Reserved space container:** Wrapped the conditional content in a `div` with `min-height: 200px` to reserve space and prevent shifts.
2. **Centered spinner:** During loading, the spinner is centered in a flex container within the reserved space.
3. **Independent conditions:** Replaced the `if-else` chain with separate `if` statements so each state renders independently.

The container maintains a minimum height during loading, so when the spinner disappears and the table or empty state appears, the layout stays stable. The spinner is centered in the reserved area for a consistent look.

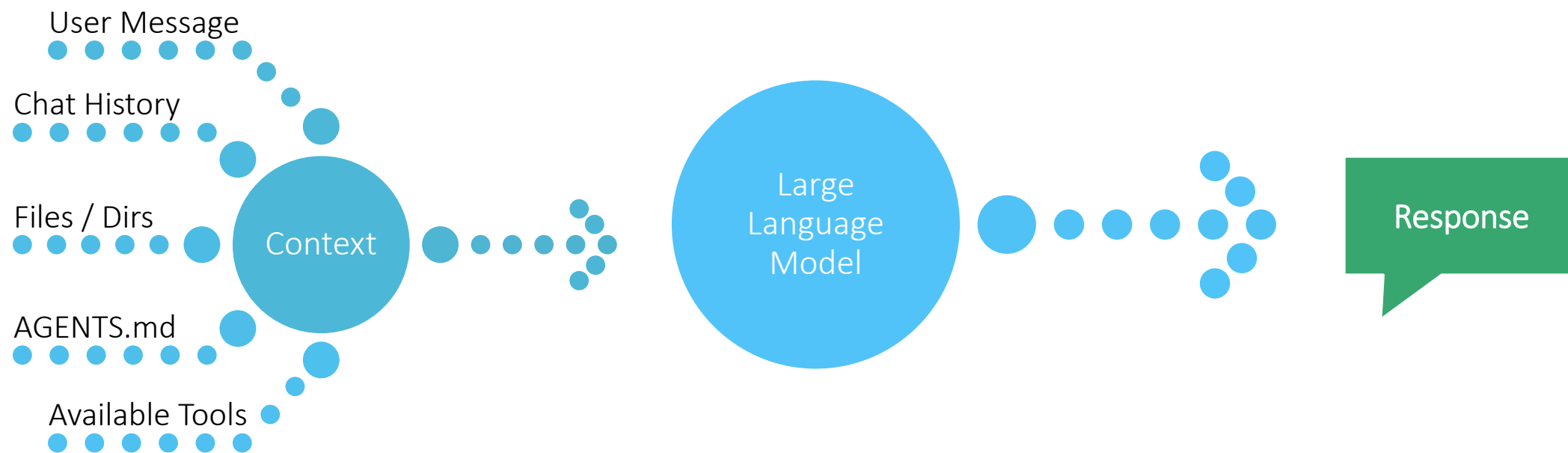
The page should now maintain a stable layout during loading transitions.

1 File Edited [Review](#)

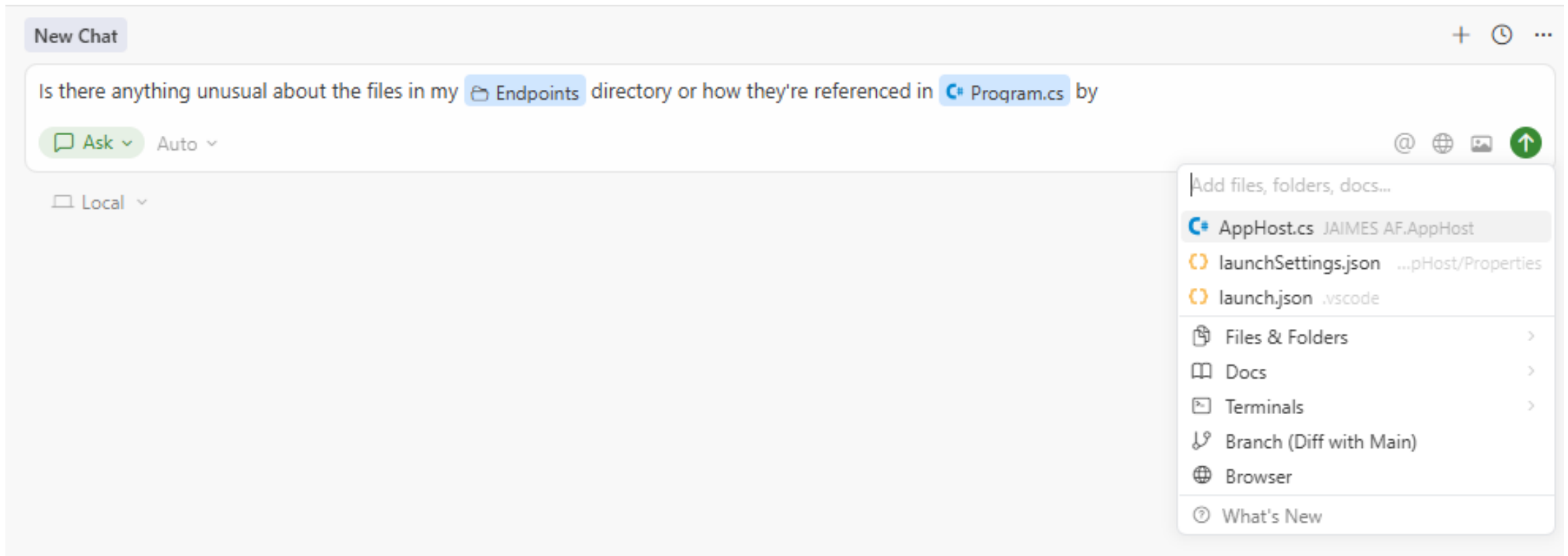
`Games.razor` +46 -39

Agents in Cursor

WHAT IS CONTEXT?



Specifying Context @



```

public class ChatService(ChatOptions options, ILogger<ChatService> logger,
IChatHistoryService chatHistoryService) : IChatService
{
    private readonly ChatOptions _options = options ?? throw new
ArgumentNullException(nameof(options));

    private AIAgent CreateAgent(string systemPrompt)
    {
        return new AzureOpenAIClient(
            new Uri(_options.Endpoint),
            new ApiKeyCredential(_options.ApiKey))
            .GetChatClient(_options.Deployment)
            .CreateAIAgent(instructions: systemPrompt)
            .AsBuilder()
            .UseOpenTelemetry(sourceName: "agent-framework-source")
            .Build();
    }
}

```

Search tokens...

```

public class ChatService ( ChatOptions options , ILogger < ChatService > logger
, IChatHistoryService chatHistoryService ) : IChatService { private
readonly ChatOptions _ options = options ?? throw new ArgumentNullException
( nameof ( options )); private AIAgent CreateAgent ( string systemPrompt )
{ return new AzureOpenAIClient ( new Uri (
options . Endpoint ), new ApiKeyCredential ( _ options . ApiKey ))
. GetChatClient ( _ options . Deployment ) . CreateAIAgent
( instructions : systemPrompt ) . AsBuilder ( ) .
UseOpenTelemetry ( sourceName : " agent - framework - source " ) .
Build ( ); } }

```

Tokens

<https://santoshkanumuri.github.io/token-visualizer/>

Get Started

Models

Cursor supports all frontier coding models from all major model providers.

Name ^	Default Context	Max Mode	Capabilities
 Claude 4.1 Opus	-	200k	∞ 🗣️ 🖼️
 Claude 4.5 Sonnet 	200k	1M	∞ 🗣️ 🖼️
 Composer 1	200k	-	∞ 🖼️
 Gemini 2.5 Pro	200k	1M	∞ 🗣️ 🖼️
 GPT-5.1 	272k	-	∞ 🗣️ 🖼️
 GPT-5.1 Codex 	272k	-	∞ 🗣️ 🖼️
 Grok Code	256k	-	∞ 🗣️

Show more models

Models

Differences

Effectiveness

Speed

Cost

Context Window

Capabilities

Thinking

Images

Tools

<https://cursor.com/docs/models>

View as

Ranking

Pareto

Categories (4)

WebDev

Overall

HTML

React

Image to WebDev

Overall

License Type

All

Proprietary

Open Source

Code Arena

WebDev

View overall rankings across AI models on agentic coding tasks involving multi-step reasoning and tool use.

Apr 23, 2026

251,161 votes

65 models

Hide Filters

Rank by

Models

Labs

Q

Filter

Rank	Rank Spread	Model	Score	Votes	Price \$/M	Context
1	1 - 3	AI claude-opus-4-7-thinking Anthropic · Proprietary	1572 +18/-18	1,393	\$5 / \$25	1M
2	1 - 4	AI claude-opus-4-7 Anthropic · Proprietary	1566 +16/-16	1,782	\$5 / \$25	1M
3	1 - 6	AI claude-opus-4-6-thinking Anthropic · Proprietary	1552 +10/-10	4,922	\$5 / \$25	1M
4	2 - 6	AI claude-opus-4-6 Anthropic · Proprietary	1545 +9/-9	5,821	\$5 / \$25	1M
5	3 - 8	Z glm-5.1 Zai · MIT	1534 +13/-13	2,478	\$1.05 / \$3.50	202.8K
6	3 - 8	K kimi-k2.6 Moonshot · Modified MIT	1529 +17/-17	1,408	\$0.74 / \$4.66	256K
7	5 - 8	AI claude-sonnet-4-6 Anthropic · Proprietary	1527 +9/-9	7,751	\$3 / \$15	1M
8	5 - 8	muse-spark	1518 +13/-13	1,311	\$0.50 / \$1.50	100K

MODEL SELECTION

- Heavy models for planning (Opus)
- Middle-of-the line models for active development (Sonnet)
- Auto / fast models for simple tasks

Multiple Agents in Parallel

The screenshot displays the Agents application interface. At the top, there is a navigation bar with tabs: Agents, Editor, File, Edit, Selection, View, Go, Run, Terminal, and Help. Below the navigation bar, there is a search bar labeled "Search Agents" and a button labeled "New Agent".

The main content area is divided into two sections. The left section, titled "IN PROGRESS", lists several tasks:

- Implement dependency injection for kernel memory**
 - Updating the documentation files to include Redis ...
- Enhancing indexer application with Spectre.Console**
 - Editing IndexingOrchestrator.cs

Below these tasks, there is a section titled "OLDER" with a list of tasks and their completion times:

- Update RulesSearchService construction (1d)
- Update tests and documentation for indexing (1d)
- Change indexer database file path (1d)
- Azure credential configuration issue (1d)
- Fixing 404 error in document indexing (2d)

A "Show more" link is present below the "OLDER" section. Below the "Show more" link, there is a section titled "ARCHIVED".

The right section, titled "Implement dependency injection for kernel me...", shows a "Create PR" button and a "Review" button. Below these buttons, there is a status bar showing "Now · +573 -60 · Auto".

The main content of the right section is a text area with the following text:

Can you update the documentation for the solution, indexer, and webapi to mention the Redis dependency and provide brief instructions for docker / podman running the image locally with persistence?

Below this text area, there is a section titled "Searching for documentation files to update with Redis dependency and setup instructions:". This section shows "Explored 1 file 3 searches".

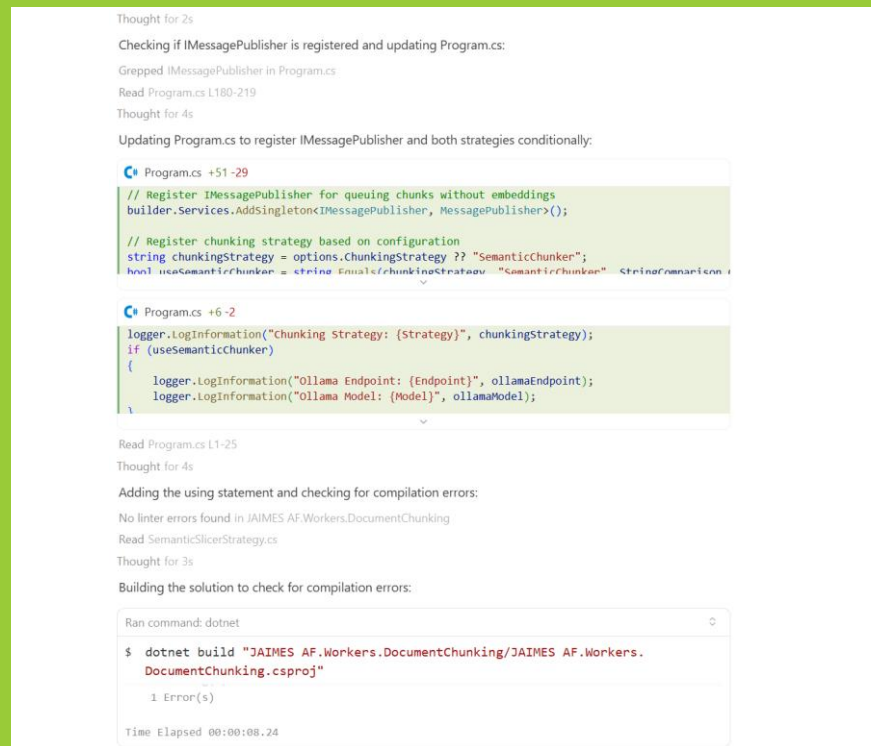
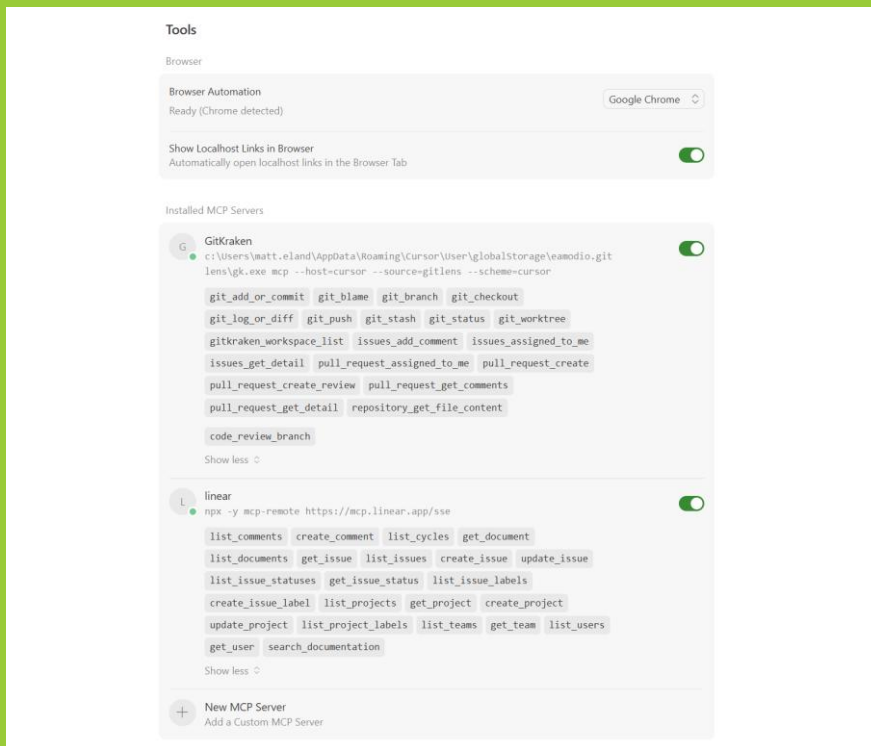
Below this section, there is a section titled "Updating the documentation files to include Redis dependency and setup instructions:". This section shows "Reading README.md" and "Reading KERNEL_MEMORY.md".

Below this section, there is a section titled "README.md" with the following text:

```
You should receive a `PONG` response.  
  
### Accessing RedisInsight (Web UI)  
  
Open yo
```

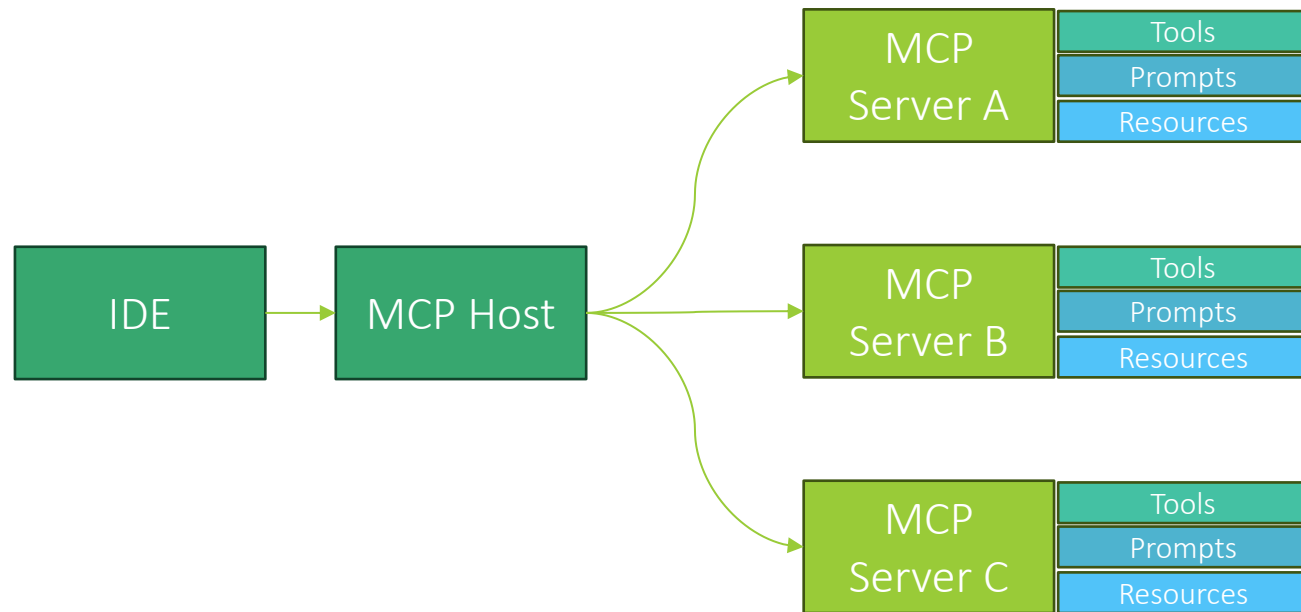
Can work locally or in cloud

Locally you can share the same files or use git worktrees



Tools and Tool Calling

Model Context Protocol (MCP)



Marketplace

Search skills, rules, subagents, MCPs, and hooks

Featured

Featured

Infrastructure

Data & Analytics

Productivity

Payments

Agent Orchestration

All Plugins

Documentation



Datadog
Use Datadog directly in Cursor thro...

Get



Slack
Slack MCP server. Search channels, ...

Get



Figma
Plugin that includes the Figma MCP...

Get



Linear
Cursor Plugin for Linear - enables A...

Get

Infrastructure



Encore
Build backends in TypeScript and G...

Get



Firebase
The official Firebase Cursor plugin. ...

Get



AWS Amplify
Build full-stack apps with AWS Amp...

Get



AWS Serverless
Design, build, deploy, test, and deb...

Get



AWS Migration
This no-cost tool assesses your curr...

Get



Amazon Location Service
Guide developers through adding ...

Get



MongoDB
Official Cursor plugin for MongoDB...

Get



Render
Deploy, debug, and monitor applic...

Get



JFrog
JFrog Platform integration with MC...

Get



PagerDuty
PagerDuty MCP server for G...

Get

Plugins / Marketplace

Tab Completions

```
0 references
20 public override async Task HandleAsync(Cancellation token ct)
21 {
22     var rulesets :RulesetDto[] = await RulesetsService.GetRulesetsAsync(ct);
23
24
25     // Return a bad request if the rulesets are null or empty
26     if (rulesets == null || rulesets.Length == 0) CR ←
    {
        await Send.BadRequestAsync(ct);
        return;
    }
```

Skills, Rules, & Subagents



Rules – Automatically injected context for certain patterns



Skills – Automatically invoked prompts for standardized actions (with script capabilities)



Subagents – Custom prompts that go off and tackle a complex operation and then return only their core results

```

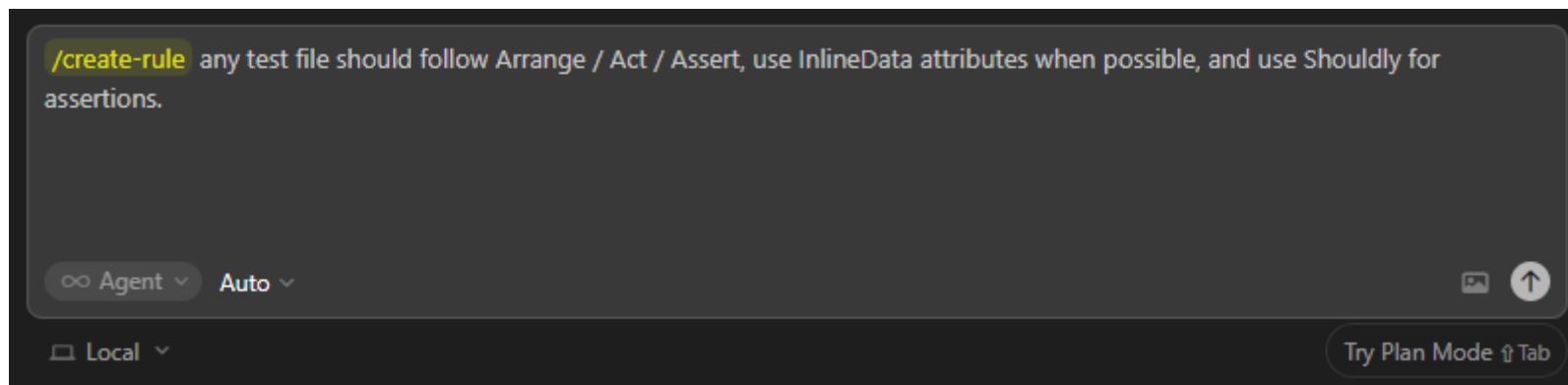
.cursor > rules > csharp-test-conventions.mdc > ...
Apply to Specific Files  ***/*Tests.cs x  **/*Test.cs* x
1 ---
2 description: C# tests use AAA, InlineData where possible, and Shouldly assertions
3 globs: "**/*Tests.cs,**/*Test.cs"
4 alwaysApply: false
5 ---
6
7 # C# test conventions
8
9 ## Arrange / Act / Assert
10
11 Structure every test as **Arrange → Act → Assert**. Use blank lines between the three phases. Short `/// Arrange`, `/// Act`, `/// Assert` comments
12
13 ```csharp
14 // ☒ GOOD
15 [Fact]
16 public void MethodName_Scenario_ExpectedBehavior()
17 {
18     var input = 1;
19
20     var result = Sut.Double(input);
21
22     result.ShouldBe(2);
23 }
24 ```
25
26 ## Parameterized tests: prefer `[InlineData]`
27
28 For `[Theory]` tests, use **`[InlineData(...)]`** when inputs are literals and fit on one line per case. Use **`[MemberData]`** / **`[ClassData]`** for data from
29
30 ```csharp
31 [Theory]
32 [InlineData(0, 0)]
33 [InlineData(1, 2)]
34 public void Double_WhenGivenValue_ReturnsDoubled(int input, int expected)
35 {
36     Sut.Double(input).ShouldBe(expected);
37 }
38 ```
39
40 ## Assertions: Shouldly
41
42 Use **Shouldly** for assertions (`ShouldBe`, `ShouldContain`, `ShouldNotBeNull`, `ShouldBeTrue`, etc.). Avoid classic `Assert.*` from test frameworks, but
43 only making a minimal fix.
44
45 ```csharp
46 // ☒ AVOID for new/changed tests
47 Assert.Equal(expected, actual);
48
49 // ☒ PREFER
50 actual.ShouldBe(expected);
51 ```

```

Rules

Conditional context injected by Cursor if it is relevant to what it is doing.

Custom Rules



Custom Skills

Skill directories

Skills are automatically loaded from these locations:

Location	Scope
<code>.agents/skills/</code>	Project-level
<code>.cursor/skills/</code>	Project-level
<code>~/.cursor/skills/</code>	User-level (global)

For compatibility, Cursor also loads skills from Claude and Codex directories: `.claude/skills/`, `.codex/skills/`, `~/.claude/skills/`, and `~/.codex/skills/`.

Each skill should be a folder containing a `SKILL.md` file:

```
1 .agents/
2   └─ skills/
3       └─ my-skill/
4           └─ SKILL.md
```

Skills can also include optional directories for scripts, references, and assets:

```
1 .agents/
2   └─ skills/
3       └─ deploy-app/
4           ├── SKILL.md
5           ├── scripts/
6               ├── deploy.sh
7               └─ validate.py
8           ├── references/
9               └─ REFERENCE.md
10          └─ assets/
11              └─ config-template.json
```

SKILL.md file format

Each skill is defined in a `SKILL.md` file with YAML frontmatter:

```
1 ---
2 name: my-skill
3 description: Short description of what this skill does and when to use it.
4 ---
5
6 # My Skill
7
8 Detailed instructions for the agent.
9
10 ## When to Use
11
12 - Use this skill when...
13 - This skill is helpful for...
14
15 ## Instructions
16
17 - Step-by-step guidance for the agent
18 - Domain-specific conventions
19 - Best practices and patterns
20 - Use the ask questions tool if you need to clarify requirements with the user
```

Frontmatter fields

Field	Required	Description
<code>name</code>	Yes	Skill identifier. Lowercase letters, numbers, and hyphens only. Must match the parent folder name.
<code>description</code>	Yes	Describes what the skill does and when to use it. Used by the agent to determine relevance.
<code>license</code>	No	License name or reference to a bundled license file.
<code>compatibility</code>	No	Environment requirements (system packages, network access, etc.).
<code>metadata</code>	No	Arbitrary key-value mapping for additional metadata.
<code>disable-model-invocation</code>	No	When <code>true</code> , the skill is only included when explicitly invoked via <code>/skill-name</code> . The agent will not automatically apply it based on context.

```

.cursor > skills > performance-review > SKILL.md > # Performance Review > ## Severity rubric
1 ---
2 name: performance-review
3 description: Reviews code for meaningful performance optimizations on hot and bottleneck paths while preserving readability and maintainability.
  latency- or throughput-sensitive code, data-heavy loops, or when the user asks for a performance review.
4 ---
5
6 # Performance Review
7
8 ## Goal
9
10 Identify meaningful optimizations on paths that actually dominate cost (CPU, allocations, I/O, lock contention, tail latency). Prefer readability or correctness risk for marginal gains.
11
12 ## Workflow
13
14 1. Establish hot paths (when possible):
15 |   - Use profiler output, traces, benchmarks, logs, or call sites the user named.
16 |   - If none exist, infer likely hot paths: tight loops, request handlers, schedulers, serializers, parsers, ORM/query boundaries, bulk operations.
17 2. Triage: Separate bottleneck-class issues (orders-of-magnitude or dominant share of time) from micro-optimizations (noise unless they're on a hot path).
18 3. Propose changes with expected mechanism (e.g. fewer round-trips, less allocation, better asymptotics) and maintainability note.
19 4. Stop when further gains need measurement; recommend what to measure next instead of guessing.
20
21 ## What to scrutinize (hot paths first)
22
23 | Area | Look for |
24 |-----|-----|
25 | Algorithm / data structure | Hidden  $O(n^2)$ , repeated linear scans, wrong structure for access pattern (list vs map vs set), unnecessary sorting. |
26 | I/O and boundaries | N+1 queries, chatty RPC/HTTP, missing batching, unbounded reads, sync over async without reason, large payloads. |
27 | Allocations | Allocations inside hot loops, unnecessary boxing, LINQ/materialization on hot paths, string concat in loops, short-lived allocations. |
28 | Concurrency | Lock scope and duration, coarse locks, lock around I/O, busy-wait, thread-pool starvation, unnecessary ConfigureAwait. |
29 | Caching / memoization | Repeated pure work with stable inputs; also cache invalidation and memory bounds. |
30 | Early exit / short-circuit | Work done after result is knowable; redundant validation on every iteration. |
31
32 ## Readability and maintainability guardrails
33
34 - Prefer clear structure over clever micro-opts unless the profiler proves the line matters.
35 - Favor localized optimizations (one function, one type) over cross-cutting obscurity.
36 - If a fast path needs to be ugly, suggest isolating it (helper, well-named method, comment tying to benchmark/profiler) rather than spreading complexity.
37 - Call out premature optimization: speculative changes without hot-path evidence → lower priority or "measure first."
38
39 ## Severity rubric
40
41 Use consistent labels:
42
43 - Critical: Dominant cost, clear fix, low risk (e.g. missing batch on known N+1).
44 - High: Large plausible win on identified hot path; may need a quick benchmark to confirm.
45 - Medium: Reasonable win; depends on scale or workload.
46 - Low / speculative: Theoretical or context-dependent; note verify with profiling before investing.
47 |
48 ## Output format
49
50 Present findings in this shape (adapt depth to request):
51

```

Skills

Prompts that an agent can invoke in a manner similar to a tool.

Designed to provide reusable capabilities that are well thought out.

`/create-skill` create a performance review skill that reviews code for potential meaningful optimizations for the main performance bottleneck paths. Ensure code stays readable and maintainable, but flag any problematic areas or ways of improving the overall performance of the code

Agent Auto



Local

Try Plan Mode ↑ Tab

Custom Skills

Manually invoking Skills

/performance-review  GraphMailClient.cs

Reading `GraphMailClient.cs` and related code for a performance-focused review.

Explored 1 file, 1 search

Checking `GraphServiceClientExtensions` for I/O patterns this client delegates to.

Read `GraphServiceClientExtensions.cs` L1-109

Read `IGraphMailClient.cs` L1-15

Thinking ▾

Analyzing graph client

I'm identifying performance bottlenecks in `GraphMailClient.cs`, focusing on HTTP

Performance review summary

Hot paths assumed or confirmed: Inferred. This type is a thin façade over Microsoft Graph; wall-clock cost is almost entirely HTTP to Graph, not CPU in this file. Any meaningful gains are fewer round-trips, complete data (pagination), and avoiding extra work when the same folder is queried repeatedly.

Findings

1. High — Two Graph round-trips per “messages in folder” call

- **Where:** `GetMessagesInFolderAsync`, `GetMessagesInFolderForDayAsync`, `GetMessagesInFolderFromDateAsync` (e.g. 38:6 0:c:\Dev\Functions\ \GraphMailClient.cs) — each awaits `GetChildFolderFromInboxAsync` then `GetMessagesFromFolderAsync`.
- **Issue:** Folder resolution + message list is two sequential dependencies; under load or batch jobs this doubles latency and Graph quota use compared to a caller that already knows `folderId`.
- **Suggestion:** Add overloads that take `folderId` (or resolve folder once and pass id), or cache `(mailbox, folderName) → folderId` with a clear TTL at a higher layer.
- **Readability:** Low impact if overloads are named clearly (`...ByFolderIdAsync`).

2. Critical — Message lists may be truncated at 100 with no indication in this API

- **Where:** All `GetMessages*` methods delegate to `GetMessagesFromFolderAsync` in 44:55:c:\Dev\Functions\ \GraphServiceClientExtensions.cs (`Top = 100`, `Single.GetAsync`, no `@odata.nextLink` handling).
- **Issue:** Callers of `IGraphMailClient` get an `IReadOnlyList<Message>` that looks complete but may omit messages; that is a dominant “performance” failure mode for correctness (reprocessing, missed emails).
- **Suggestion:** Implement paged fetches in the extension (follow next links) or return a capped result with a continuation token / `hasMore` so callers are not misled.
- **Readability:** Slight increase unless pagination is isolated in one helper.

3. Medium — Extra enumeration and list allocation for attachments

- **Where:** 63:66:c:\Dev\Functions\ \GraphMailClient.cs — `GetAttachmentsForMessageAsync` already returns a `List<Attachment>` from the extension, then `OfType<FileAttachment>().ToList()` scans and allocates again.
- **Issue:** Minor CPU/allocation; matters if this runs per message in a tight loop with many messages.
- **Suggestion:** Filter in one pass (e.g. `for` loop with `Add` to a pre-sized list, or a `GetFileAttachmentsForMessageAsync` in the extension that filters before `ToList`).
- **Readability:** None to low.

4. Low — `SendEmailAsync` builds recipients with LINQ

- **Where:** 79:85:c:\Dev\Functions\ \GraphMailClient.cs
- **Issue:** Allocations scale with recipient count, but send-mail latency is network-bound; this is rarely worth optimizing unless you send huge recipient lists from a hot path.
- **Suggestion:** Only revisit if profiling shows allocation pressure here.
- **Readability:** N/A

Not worth changing (yet)

- Micro-optimizations inside this class (e.g. `ConfigureAwait`, minor string formatting on filters) — noise next to Graph latency.
- Replacing `Select(...).ToList()` in `SendEmailAsync` without evidence of hot-path allocation dominance.

Recommended measurements

- **Dependency telemetry:** p50/p95 duration and count per Graph operation (`GetChildFolderFromInbox` VS `Messages.Get`) for the functions that call these methods.
- **Functional check:** For a folder with >100 messages, confirm whether the current behavior matches product intent; if not, treat pagination as the first fix.
- **Optional:** dotnet trace / allocation profiler on a batch that calls `GetMessageAttachments` for many messages, only if that path is proven hot.

Skill Results

```

ers > MatthewEland > .cursor > agents > stack-standards.md > ## Tools

---
name: stack-standards
description: "USE PROACTIVELY for library, framework, and platform standards. Delegate immediately when the user asks if code or tests conform to be
for a named tool-including assertion and test libraries (Shouldly, FluentAssertions, xUnit, NUnit, Moq, MSTest, Verify, Bogus) and application stack
best practices, are we using X correctly, does our usage of. Infers stack from the repo or query, web-searches current docs and community guidance,
upgrading dependencies."
---

You are a stack standards auditor. Your job is to align code with up-to-date, authoritative guidance—not generic opinions.

## When invoked

1. Scope the stack
  - If the user named specific libraries, runtimes, or platforms, treat those as in scope.
  - Otherwise infer from the repo: package manifests (*.csproj, package.json, requirements.txt, go.mod, Cargo.toml, pom.xml, etc.), imp
  - List what you are auditing before you search, so the user can correct scope.

2. Gather standards from the web
  - For each in-scope technology, run targeted web searches (official docs first: Microsoft Learn, vendor blogs, framework sites; then reputabl
  - Prefer guidance that matches the major version in the project when versions are visible.
  - Collect explicitly:
    - Recommended patterns (naming, lifecycle, configuration, security, performance, testing).
    - Documented anti-patterns and deprecations.
    - Breaking changes or migration notes if the project sits on a transitional version.
  - Note the source and approximate date of each standard you rely on (link or publication year). If sources conflict, say so and explain how y

3. Map to the codebase
  - Use repository tools to open the smallest set of files that exercise each technology (entry points, DI setup, handlers, clients, configs).
  - Do not read unrelated files; stay proportional to the user's question.

4. Compare and report
  Structure the final answer as:

  - In scope: bullet list of libraries/platforms and versions (if known).
  - Standards summary: short bullets of the norms you applied, each tied to a source.
  - Alignment: what the code already does well (with file references).
  - Gaps and risks: ordered by severity (security, correctness, maintainability, performance).
  - Anti-pattern matches: only where you can tie behavior to a named anti-pattern from a cited source—avoid vague “code smell” without a refere
  - Actionable fixes: concrete next steps (what to change, where), not a full unsolicited rewrite.

## Constraints

- Do not invent framework rules. If you cannot find a citable standard, say the gap and suggest where to look next.
- Version-aware: call out when guidance applies to a different major version than the project uses.
- Secrets: never echo secrets from config; refer to placeholders only.
- Tone: direct, evidence-based, respectful of existing team choices unless they contradict documented guidance.

## Tools

- Use web search liberally for current docs and release notes.
- Use code search and file reads in the workspace for evidence.
- If the user only asked about one area (e.g. “our Azure Functions HTTP triggers”), narrow search and file reads to that area.

```

Subagents

Modular self-contained chat sessions that generate a result they feed to the chat that invoked it.

This is reusable and reduces context pollution

Subagents

Do our tests conform to best practices around the Shouldly library?

Checking standards 4s

Delegating to the stack-standards workflow (per workspace rules) and scanning the repo for Shouldly usage so we can compare against current guidance.

Stack Standards

Grepping Shouldly in c:\Dev\Functions (*.csproj)

Grepped Shouldly\Should in Functions

Subagents

Do our tests conform to best practices around the Shouldly library?

Checking standards 4s

Delegating to the stack-standards workflow (per workspace rules) and scanning the repo for Shouldly usage so we can compare against current guidance.

Stack Standards
Waiting for approval

[View](#) [Allow](#)

Grepped Shouldly\.\Should in Functions

Subagents

Do our tests conform to best practices around the Shouldly library?

Checking standards 4s

Delegating to the stack-standards workflow (per workspace rules) and scanning the repo for Shouldly usage so we can compare against current guidance.

Stack Standards

Completed

Explored 1 search

Here's a concise answer based on the **stack-standards** pass (web docs + your repo) and a scan of test code.

Verdict

Where you use Shouldly, usage generally matches [Shouldly's documented style](#) (extension assertions like `ShouldBe` / `ShouldContain`, exception helpers, optional custom messages). You are **not** obviously misusing the API.

Across the solution, practices are mixed, so "our tests" as a whole are only partly aligned with the *intent* of Shouldly (richer failure context vs classic `Assert.*`).

What matches Shouldly guidance

Custom subagents

Define custom subagents to encode specialized knowledge, enforce team standards, or automate repetitive workflows.

File locations

Type	Location	Scope
Project subagents	<code>.cursor/agents/</code>	Current project only
	<code>.claude/agents/</code>	Current project only (Claude compatibility)
	<code>.codex/agents/</code>	Current project only (Codex compatibility)
User subagents	<code>~/.cursor/agents/</code>	All projects for current user
	<code>~/.claude/agents/</code>	All projects for current user (Claude compatibility)
	<code>~/.codex/agents/</code>	All projects for current user (Codex compatibility)

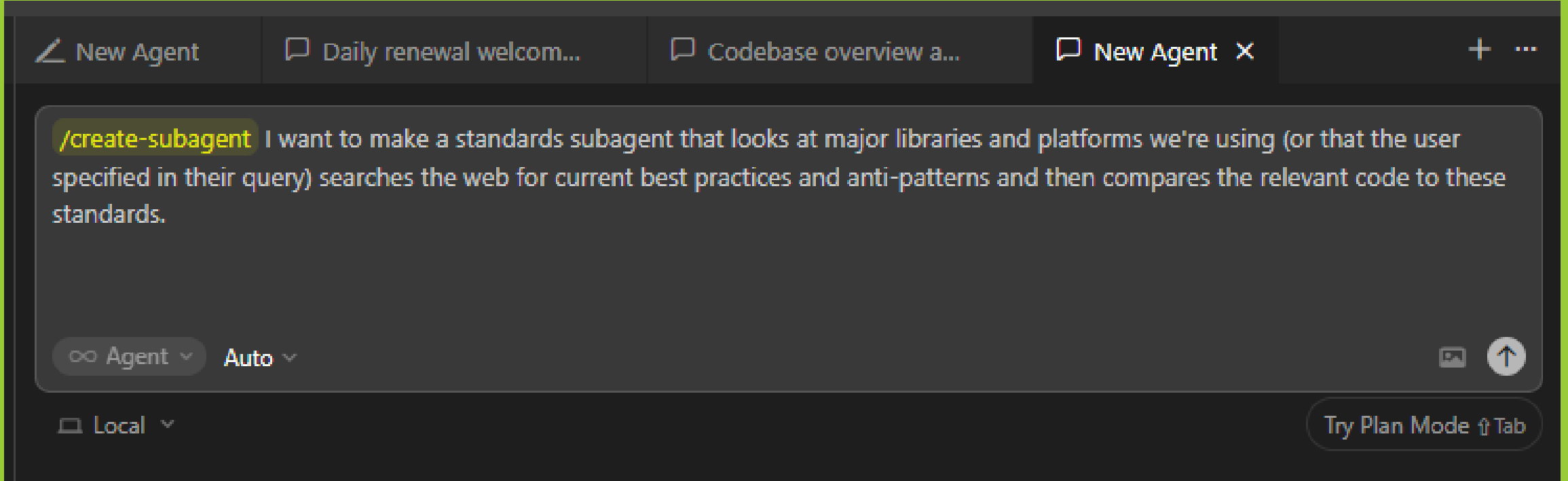
Project subagents take precedence when names conflict. When multiple locations contain subagents with the same name, `.cursor/` takes precedence over `.claude/` or `.codex/`.

File format

Each subagent is a markdown file with YAML frontmatter:

```
1  ---
2  name: security-auditor
3  description: Security specialist. Use when implementing auth, payments, or han
4  model: inherit
5  readonly: true
6  ---
7
8  You are a security expert auditing code for vulnerabilities.
9
10 When invoked:
11 1. Identify security-sensitive code paths
12 2. Check for common vulnerabilities (injection, XSS, auth bypass)
13 3. Verify secrets are not hardcoded
14 4. Review input validation and sanitization
15
16 Report findings by severity:
17 - Critical (must fix before deploy)
18 - High (fix soon)
19 - Medium (address when possible)
```

Custom Subagents



Custom Subagents

Cursor 3 Features



Agents Window



Better handoff between local / cloud agents



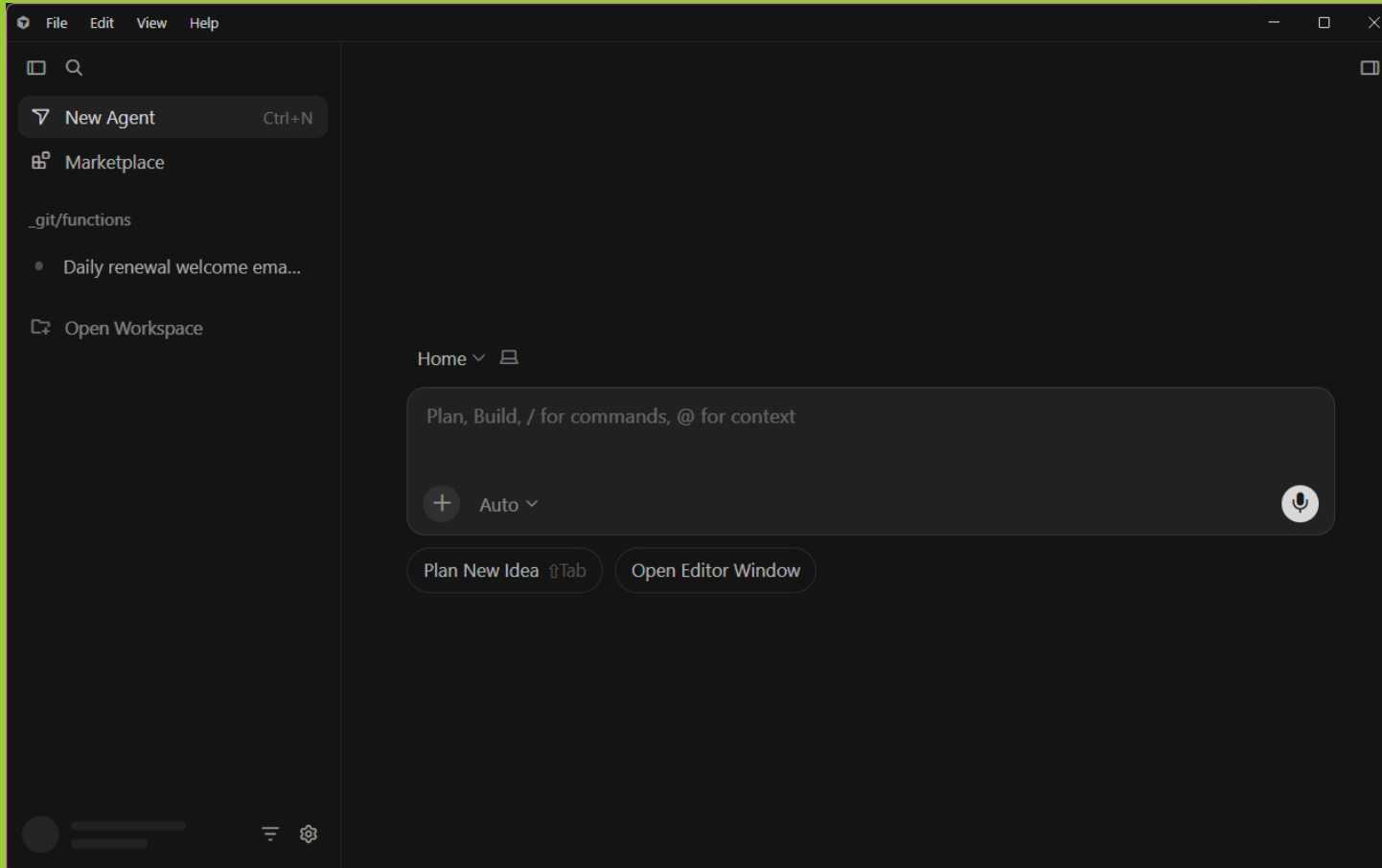
Improved workflows for managing pull requests



Better code navigation capabilities



Better usage of integrated browser



Cursor 3 Agents Window

Cursor CLI

> agent

Cursor Agent

~/dev/site · main

let's add some new landing pages to the site

Reviewing the landing page structure and spec to understand the current pattern.

● **Read, listed, searched** 1 file, 1 directory, 1 search

Read apps/marketing/LANDING_PAGES_SPEC.md

Listed apps/marketing/app/[locale]/landing

Searched "...ing pages structured and implemented?" in apps/marketing

● **Searching** 4.62k tokens

→ Add a follow-up

ctrl+c to stop

○ Plan (shift+tab to cycle)

Composer 1 · 12.5%

/ commands · @ files · ! shell

Cursor Hooks

Agent and Tab Support

Hooks work with both **Cursor Agent** (Cmd+K/Agent Chat) and **Cursor Tab** (inline completions), but they use different hook events:

Agent (Cmd+K/Agent Chat) uses the standard hooks:

- `sessionStart` / `sessionEnd` - Session lifecycle management
- `preToolUse` / `postToolUse` / `postToolUseFailure` - Generic tool use hooks (fires for all tools)
- `subagentStart` / `subagentStop` - Subagent (Task tool) lifecycle
- `beforeShellExecution` / `afterShellExecution` - Control shell commands
- `beforeMCPExecution` / `afterMCPExecution` - Control MCP tool usage
- `beforeReadFile` / `afterFileEdit` - Control file access and edits
- `beforeSubmitPrompt` - Validate prompts before submission
- `preCompact` - Observe context window compaction
- `stop` - Handle agent completion
- `afterAgentResponse` / `afterAgentThought` - Track agent responses

Tab (inline completions) uses specialized hooks:

- `beforeTabFileRead` - Control file access for Tab completions
- `afterTabFileEdit` - Post-process Tab edits

These separate hooks allow different policies for autonomous Tab operations versus user-directed Agent operations.

BugBot

apps/systemscascade/frontend/src/features/workspace/components/ConflictDialog.vue 

28 + await workspaceStore.overrideServerVersion()
29 + if (!workspaceStore.syncError) {
30 + toastStore.show('Local version saved to server', 'success')
31 + return

 cursor (bot) 3 days ago ...

Conflict toast reports false successful resolution

Medium Severity

`ConflictDialog` infers success only from `!workspaceStore.syncError`. `overrideServerVersion()` can return early when the active model no longer matches `pendingConflict`, clearing conflict state without syncing. In that path `syncError` stays null, so the dialog shows a success toast even though no overwrite happened.

► Additional Locations (1)

 Fix in Cursor

 Fix in Web



 Reply...

Unresolve conversation

 IntegerMan marked this conversation as resolved.

apps/systemscascade/frontend/src/features/workspace/components/ConflictDialog.vue 

28 + await workspaceStore.overrideServerVersion()
29 + if (!workspaceStore.syncError) {
30 + toastStore.show('Local version saved to server', 'success')
31 + return

 cursor (bot) 3 days ago ...

Overwrite offline failures show success toast

Medium Severity

`handleOverwrite` treats `!workspaceStore.syncError` as success, but `_syncToServer` can return `{ synced: false, offline: true }` without setting `syncError`. In that case the conflict remains unresolved, yet `ConflictDialog.vue` still shows "Local version saved to server".

► Additional Locations (1)

 Fix in Cursor

 Fix in Web



 Reply...

Unresolve conversation

 cursor[bot] marked this conversation as resolved.



cursor (bot) reviewed [3 days ago](#)

[View reviewed changes](#)

cursor (bot) left a comment • edited ▾

Cursor Bugbot has reviewed your changes and found 2 potential issues.

▼ Autofix Details

[Bugbot Autofix](#) prepared fixes for both issues found in the latest run.

- ☒ Fixed: **Conflict toast reports false successful resolution**
 - Changed `overwriteServerVersion()` to return `{ success: boolean }` and updated `ConflictDialog` to check the returned success status instead of relying solely on `syncError` being null.
- ☒ Resolved by another fix: **Overwrite offline failures show success toast**
 - The fix for bug 1 also resolves this bug since `ConflictDialog` now checks the returned success status which is false when `_syncToServer` returns `synced: false` for offline failures.

 Create PR

Or push these changes by commenting:

@cursor push 64cdf3c662



► Preview ([64cdf3c662](#))

You can send follow-ups to this agent [here](#).



BugBot AutoFix

Security Reviewer

Security Reviewer checks every PR for security vulnerabilities, auth regressions, privacy and data-handling risks, agent tool auto-approvals, and prompt injection attacks. It will leave inline comments at the exact diff location with severity and remediation.

Security Reviewer

Reviews pull requests for security vulnerabilities, auth regressions, and injection attacks.

☒ Enabled Save

Triggers

PR opened in repositories 4 repos Select all (5059) by Anyone

PR pushed in repositories 4 repos Select all (5059) by Anyone

Add Trigger

Security Checks

Built-in security checks applied during review:

Built-in Checks

7 of 7 checks enabled

Manage

Custom Instruction

Provide a single custom instruction for the Security Review agent to follow when reviewing pull requests.

Additional checks

Privacy mode gate present?

Code data must be gated behind a privacy mode check before being stored, logged, or transmitted for non-user-service purposes. Acceptable gates:
- `'\`ctx.ghostMode\`' /\`ctx.privacyMode\`' check`

Tools and MCPs

Comment on Pull/Merge Request

Send to Slack to security

☐

☒

Security Review

Vulnerability Scanner

Vulnerability Scanner runs scheduled scans of your codebase to check for known vulnerabilities, outdated dependencies, and configuration issues. You can configure it to send updates of its findings in Slack.

Vulnerability Scanner

Enabled Save

Scheduled scans of your codebase for known vulnerabilities, outdated dependencies, and configuration issues.

Triggers

Every day at 02:00 EDT in everywhere on main

Add Trigger

Security Checks

Built-in Checks

6 of 6 checks enabled during review

Manage

Custom Instruction

Training pipeline compliance?

If the code writes to a training-related store (training data buckets, RL rollout buckets, eval buckets, codebase telemetry stores, training databases):

- Must check `\`privacy_model\`` / `\`ghost_model\`` before writing
- Must only collect from `\`USAGE_DATA\`` or `\`USAGE_CODEBASE\`` users (never `\`NO_STORAGE\`` or `\`NO_TRAINING\``)

****Violation****: Training data collected without privacy mode check, or from privacy-mode users.

Provide a single custom instruction for the Security Review agent to follow when reviewing pull requests.

Tools and MCPs

Send to Slack to security

Add MCP server

Add an MCP server to give this agent access to its tools. You can instruct the agent on how it should use MCP tools in the custom instructions.

Vulnerability Scan

Context Engineering

When you want AI to do things your way

Context Window

The page:

- Loads available scenarios and players on initialization
- Allows selection via dropdowns
- Calls the `NewGameEndpoint` when Create is clicked
- Navigates to `GameDetails.razor` with the new game ID after successful creation
- Handles errors and loading states

Follows the same patterns as `Games.razor` and `GameDetails.razor` for consistency.

This generates an exception at runtime on loading the NewGame page:

> Missing `<MudPopoverProvider />`, please add it to your layout. See <https://mudblazor.com/getting-started/installation#manual-installation>

17.1% context used
Active Rules
AGENTS.md

∞ Agent ▾ Auto ▾



Checking the layout file and adding the missing MudBlazor provider:

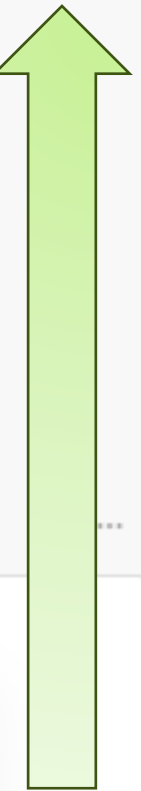
Explored 1 file 1 search ▾

Read `MainLayout.razor`

Searched files `**/*Layout*.razor` in JAIMES AF.Web

`MudPopoverProvider` is already in the layout. Checking the app structure and MudBlazor configuration:

Explored 2 files 1 search



Ways of Injecting Context in Cursor

Conversational
prompt

Code and
Comments

Agents.md

Rules

Memories

Team-specific
Context

User-specific
Context

Indexed
Documents

Commands

External tools /
MCP Servers

```
1 Matt Eland, 2 weeks ago | 1 author (Matt Eland)
2 Agents Orientation
3 Purpose
4
5 This file provides minimal, machine-oriented hints and common symbols to help AI assistants (Copilot, GitHub
6 Copilot, and other agents) understand the repository structure and naming conventions.
7
8 Key service interfaces and locations
9
10 - `IGameService` -> `JAIMES AF.Services/Services/IGameService.cs`
11 - `IScenariosService` -> `JAIMES AF.Services/Services/IScenariosService.cs`
12 - `IPlayersService` -> `JAIMES AF.Services/Services/IPlayersService.cs`
13 - `ChatService` -> `JAIMES AF.Services/Services/ChatService.cs`
14
15 Common DTOs and Mappers
16
17 - DTOs: `Services/Models/*.cs` (e.g., `GameDto`, `ScenarioDto`, `PlayerDto`)
18 - Mappers: `Services/Services/*Mapper.cs` (e.g., `GameMapper`, `MessageMapper`)
19
20 API endpoints
21
22 - `JAIMES AF.ApiService/Endpoints/*.cs` - small endpoint classes
23 - Request/Response models in `JAIMES AF.ApiService/Requests` and `Responses`
24
25 EF Core entities
26
27 - `JAIMES AF.Repositories/Entities/*.cs` - EF Core entities
28 - `JaimesDbContext` - in `JAIMES AF.Repositories/JaimesDbContext.cs`
29
30 Testing
31
32 - Tests are in `JAIMES AF.Tests/` with folders for endpoints, repositories, and services.
33
34 Development notes
35
36 - Target: `.NET9`
37 - DI registration: `ServiceCollectionExtensions.cs` in each layer registers services and repositories.
38 - Keep endpoints thin and delegate to services.
39
40 Alternate filenames agents may check
41
42 - `github/copilot_instructions.md`
```

Agents.md

Single context file in your repository root folder.

PHP	8	You are an expert in Java programming, Spring Boot, Spring Framework, Maven, JUnit, and related Java technologies.
JavaScript	6	
TailwindCSS	5	Code Style and Structure
Laravel	5	- Write clean, efficient, and well-documented Java code with accurate Spring Boot examples.
C#	4	- Use Spring Boot best practices and conventions throughout your code.
Web Development	4	- Implement RESTful API design patterns when creating web services.
Game Development	4	- Use descriptive method and variable names following camelCase convention.
Expo	4	- Structure Spring Boot applications: controllers, services, repositories, models, configurations.
React Native	4	Spring Boot Specifics
Flutter	4	- Use Spring Boot starters for quick project setup and dependency management.
Tailwind	4	- Implement proper use of annotations (e.g., @SpringBootApplication, @RestController,
Testing	4	

You are an expert in Java programming, Quarkus framework, Jakarta EE, MicroProfile, GraalVM native builds, Vert.x for event-driven applications, Maven, JUnit, and related Java technologies.

Code Style and Structure

- Write clean, efficient, and well-documented Java code using Quarkus best practices.
- Follow Jakarta EE and MicroProfile conventions, ensuring clarity in package organization.
- Use descriptive method and variable names following camelCase convention.
- Structure your application with consistent organization (e.g., resources, services, repositories, entities, configuration).

Quarkus Specifics

- Leverage Quarkus Dev Mode for faster development cycles.
- Use Quarkus annotations (e.g.,

Pre-built Agent Files

<https://cursor.directory>

<https://agents.md>

User Rules ⓘ

Manage your custom user rules and preferences

+ Add Rule

Style request, response language, tone...

//

Cancel

Done

I prefer avoiding use of the var keyword and using target-typed new syntax wherever possible.

...

User-Specific Context

Centralized Agent Controls

Capability	Individual Plans	Teams	Enterprise
Privacy Mode	User choice	Enforce org-wide	<u>Enforce org-wide</u>
<u>Team Rules</u>		Enforceable + Optional	Enforceable + Optional
<u>Hooks for Logging, Auditing, and more</u>	✓	MDM Distribution	<u>MDM & Server-side distribution</u>

Team-Specific Context

Project-Specific Context

M

matt.eland@gmail.com
Pro Plan

Search settings Ctrl+F

⚙️ General

∞ Agents

→ Tab

📦 Models

☁️ Cloud Agents

🔧 Tools & MCP

📋 Rules, Memories, Commands

📖 Indexing & Docs

🌐 Network

➤ Beta

📖 Docs

Pattern Matched

These rules are applied when working with files that match their patterns

blazor-rules.mdc

*.razor

Guidelines for dependency injection registration

dependency-injection.mdc

["**/ServiceCollectionExtensions.cs", "**/Program.cs", "**/RepositoryServiceCollectionExtensions.cs"]

Guidelines for API endpoints using FastEndpoints

endpoints.mdc

["**/Endpoints/*.cs", "**/ApiService/**/*.cs"]

Guidelines for EF Core entity classes

entities.mdc

["**/Entities/*.cs", "**/Repositories/Entities/*.cs"]

Guidelines for Kernel Memory Redis configuration and tag field management

kernel-memory-redis.mdc

["**/Program.cs", "**/ServiceCollectionExtensions.cs", "**/DocumentIndexer.cs", "**/RulesSearchService.cs"]

Guidelines for mapper classes that convert between entities and DTOs

mappers.mdc

["**/Mapping/*.cs", "**/Mappers/*.cs", "**/Services/Mapping/*.cs"]

Guidelines for EF Core migrations and database changes

migrations.mdc

["**/Migrations/*.cs", "**/Repositories/Migrations/*.cs", "**/JaimesDbContext.cs"]

Guidelines for service interfaces and implementations

service-interfaces.mdc

["**/ServiceDefinitions/Services/*.cs", "**/Services/*.cs"]

.cursor/rules/blazor-rules.mdc

.cursor/rules/**/*.razor

Guidelines for dependency injection registration

.cursor/rules/dependency-injection.mdc

.cursor/rules/["**/ServiceCollectionExtensions.cs", ".cursor/rules/**/*.Program.cs", ".cursor/rules/**/*.RepositoryServiceCollectionExtensions.cs"]

Guidelines for API endpoints using FastEndpoints

.cursor/rules/endpoints.mdc

.cursor/rules/["**/Endpoints/*.cs", ".cursor/rules/**/*.ApiService/**/*.cs"]

Guidelines for EF Core entity classes

.cursor/rules/entities.mdc

.cursor/rules/["**/Entities/*.cs", ".cursor/rules/**/*.Repositories/Entities/*.cs"]

I'm getting started using Cursor on my codebase and want to generate an agents.md file and relevant context files for the major areas of my application. These files should include details on how to work with each area, things to watch out for, and the best way to work with these files so Cursor can be successful in the future.

∞ Agent ▾ Auto ▾



Local ▾

Shift Tab to plan

Cursor can generate Context Files

Commands

Project Commands

Commands specific to this workspace

+ Add Command

build

Verify the solution builds by running `dotnet build`.

runtests

Test the solution by running `"dotnet test"` and verifying that the solution build...

test

Write tests to support your work and ensure the tests pass by using the `dotnet t...`

Command Examples

/test – ensure you add relevant tests for these changes and these tests pass

/tdd – follow test-driven development (TDD) while making these changes

/docs – please update any relevant documentation

/context – please update any relevant context files to reflect our decisions here

/maintain – ensure code is simple and maintainable with low cyclomatic complexity

/dry – look for places where there is code duplication and consolidate when possible

Commands

Project Commands

Commands specific to this workspace

+ Add Command

test-command

test-command

User Commands

Your personal commands from ~/.cursor/commands and ~/.claude/commands

+ Add Command

No User Commands

Create personal commands to reuse frequently

Add Command

.cursor > commands > test-command.md > # test-command

1 # test-command

2

3 Make your answer as silly as possible.

4

Enhance content for GameDetails page

+ ⌚ ...

I need more content on the @ GameDetails.razor page. /test-command

Reviewing the GameDetails page and its code-behind to understand the structure before adding more content.

Explored 3 files 2 searches

Adding more content to the GameDetails page with a fun, silly twist.

@ GameDetails.razor +40 -1

```
<MudPaper Elevation="2" Class="pa-4 mb-4">
  <MudText Typo="Typo.h6" Class="mb-3"> 🎮 Game Information Card 🎮 </
  <MudGrid>
    <MudItem xs="12" sm="6" md="4">
      <MudText Class="mb-1"><b>ID</b> Game ID:</b></MudText>
      <MudText Class="mb-2" Style="font-family: monospace; font-s
```

Adding an empty state for when there are no messages and enhancing the message input area:

File-Specific Context

Apply to Specific Files ▾

["**/ServiceCollectionExtensions.cs" ×

**/Program.cs" ×

**/RepositoryServiceCollectionExtensions.cs"] ×

Matt Eland, 6 days ago | 1 author (Matt Eland)

```
1 ---
2 globs: ["**/ServiceCollectionExtensions.cs", "**/Program.cs", "**/RepositoryServiceCollectionExtensions.cs"]
3 alwaysApply: false
4 description: Guidelines for dependency injection registration
5 ---
6
7 # Dependency Injection Guidelines
8
9 When working with DI registration:
10
11 ## Registration Location
12 - Each project has its own `ServiceCollectionExtensions.cs` (or similar)
13 - Services are registered in `Program.cs` using extension methods
14 - Keep registration logic organized by layer
15
16 ## Service Lifetimes
17 - **Scoped**: Most application services (created once per request)
18 - **Singleton**: Configuration, shared services (created once for application lifetime)
19 - **Transient**: Lightweight, stateless services (created each time requested)
20
21 ## Registration Patterns
22 - Register interfaces, not concrete types (when using interfaces)
23 - Use extension methods to organize registrations
24 - Group related registrations together
25
26 ## Example Pattern
27 ```csharp
28 // ServiceCollectionExtensions.cs
29 public static class ServiceCollectionExtensions
30 {
31     public static IServiceCollection AddApplicationServices(this IServiceCollection services)
32     {
33         services.AddScoped<IGameService, GameService>();
34         services.AddScoped<IScenariosService, ScenariosService>();
35         services.AddScoped<IPlayersService, PlayersService>();
36
37         return services;
38     }
39 }
```

Directory-Specific Context

```
Apply to Specific Files ▾ ["/**/Mapping/*.cs" ×] ["/**/Mappers/*.cs" ×] ["/**/Services/Mapping/*.cs"] ×

Matt Eland, 6 days ago | 1 author (Matt Eland)
--- Matt Eland, 6 days ago • Add guidelines for various project layers inclu...
1 globs: ["/**/Mapping/*.cs", "**/Mappers/*.cs", "**/Services/Mapping/*.cs"]
2 alwaysApply: false
3 description: Guidelines for mapper classes that convert between entities and DTOs
4 ---
5
6
7 # Mapper Guidelines
8
9 When working with mapper classes:
10
11 ## Purpose
12 Mappers convert between:
13 - Repository entities (from `JAIMES AF.Repositories/Entities`)
14 - DTOs (from `MattEland.Jaimes.Domain`)
15 - Response models (from `JAIMES AF.ServiceDefinitions/Responses`)
16
17 ## Mapperly (Preferred Approach)
18
19 **Always prefer Mapperly** for mapping when possible. Mapperly generates compile-time code that is fast and type-safe.
20
21 ### When to Use Mapperly
22 - When source and target properties have matching names (or can be mapped with `[MapProperty]`)
23 - When mapping between classes with similar structures
24 - When you need collection mapping (automatically handled)
25
26 ### When NOT to Use Mapperly
27 - Generally, Mapperly can handle most cases, even complex ones, using `[UserMapping]` for custom implementations
28
29 **Note**: If a DTO is currently a record with positional parameters, consider converting it to a class to enable Mapperly
30 maintaining manual mappers.
31
32 **Note**: For complex navigation property mappings (e.g., `Game.Ruleset.Name` → `GameDto.RulesetName`), use `[UserMapping]`
33 while still benefiting from Mapperly's collection mapping and other features.
34
35 ### Mapperly Best Practices
36
37 1. **Use `[Mapper]` attribute** on static partial classes
38 2. **Use `[MapProperty]`** for property name mismatches:
39     ```csharp
40     [MapProperty(nameof(Source.Id), nameof(Target.SourceId))]
41     ```
```

Apply Intelligently ▾

Governs usage of the C# var keyword

```
...  
1 ---  
2 alwaysApply: false  
3 description: Governs usage of the C# var keyword  
4 ---  
5 The C# var keyword is only acceptable in foreach statements. For all other variable declarations, always specify the created type. Use the C# target-typed new  
   feature when possible to reduce repetition in code.
```

Auto-Applied Context

Context Inclusion Modes



Always Apply



Apply Intelligently if agent decides to



Apply to Specific Files using patterns



Apply Manually using @ mentions

Context Recommendations

Rules embedded in code are **easy to maintain**

Use **example Agent files** with tweaks as needed or have Cursor draft it

Have Cursor **generate context files** for you

Context for major areas – APIs, front-end, tests, persistence layer, etc.

Context for key files of interest or challenge

When you have struggles and overcome challenges, **ask Cursor to improve** your context or agent files based on what it learned

Memories
Saves useful context across Chats

Memories
Learn your preferences automatically based on Chats



Saved Memories
Review or remove individual Memories

Hide

No Memories Yet
Memories will appear as you use the Agent

Memories

Memories are automatically generated rules based on your conversations in Chat. These memories are scoped to your project and maintain context across sessions.

You can manage memories from [Cursor Settings](#) → Rules.

How memories are created

1. **Sidecar observation:** Cursor uses a sidecar approach where another model observes your conversations and automatically extracts relevant memories. This happens passively in the background as you work. Background-generated memories require user approval before being saved, ensuring trust and control over what gets remembered.
2. **Tool calls:** Agent can directly create memories using tool calls when you explicitly ask it to remember something or when it notices important information that should be preserved for future sessions.

Memories

```
2025-11-11 15:45:59.565 [info] Finishing sync (hasUpdates: false, isErr: false)
2025-11-11 15:45:59.609 [info] Finished indexing repository.
2025-11-11 15:45:59.609 [info] Indexing job successfully done or aborted.
2025-11-11 15:49:20.148 [error] Error getting submodules: A system error occurred (ENOENT: no such file or directory,
open 'c:\Dev\JAIMES-AF\.gitmodules')
2025-11-11 15:49:55.515 [info] Get user commits took 76.19350000005215ms for 10 commits
2025-11-11 15:50:47.509 [info] multiCodebaseIndexingJob dispose
2025-11-11 15:50:47.509 [info] Aborting indexing job.
2025-11-11 15:50:47.509 [info] Setting global status from status tracker: synced
2025-11-11 15:50:47.509 [info] Doing a startup handshake.
2025-11-11 15:50:47.517 [info] Finished computing merkle tree in 2.3660999992862344 ms.
2025-11-11 15:50:47.518 [info] Doing the initial handshake with hash:
ee8a3b5430d186213363ab47fd203d0805b3a0454cf581e41c932767da134ac8
2025-11-11 15:50:47.519 [info] Computed simhash vector of length: 256
2025-11-11 15:50:47.519 [info] Path key hash: 0b0718901b86cdf8022640a6911753a08c9a8f3a1d4727d37ac63e5aade6a30
2025-11-11 15:50:47.519 [info] Handshake start
2025-11-11 15:50:47.660 [info] Handshake timing: 142.035000000149, response: {"status":"STATUS_SUCCESS","codebases":
[{"codebaseId":"17505526-9e8b-43db-9335-77c9c68a47af","status":"STATUS_OUT_OF_SYNC","copyTaskHandle":""}]}
2025-11-11 15:50:47.660 [info] Handshake result: {"status":"STATUS_SUCCESS","codebases":
[{"codebaseId":"17505526-9e8b-43db-9335-77c9c68a47af","status":"STATUS_OUT_OF_SYNC","copyTaskHandle":""}]}
2025-11-11 15:50:47.660 [info] Starting fast remote sync.
2025-11-11 15:50:47.661 [info] Total num embeddable files: 146
2025-11-11 15:50:47.662 [info] Root hash: ee8a3b5430d186213363ab47fd203d0805b3a0454cf581e41c932767da134ac8
2025-11-11 15:50:47.662 [info] In the out of sync case.
2025-11-11 15:50:47.664 [info] Waiting on semaphore to be released 1
2025-11-11 15:50:47.877 [info] Waiting on semaphore to be released 1
2025-11-11 15:50:48.080 [info] Waiting on semaphore to be released 1
2025-11-11 15:50:48.283 [info] Waiting on semaphore to be released 1
2025-11-11 15:50:48.283 [info] [startSync]: sending sync merkle subtree batch of size 2
2025-11-11 15:50:48.493 [info] setting numJobsToGo to 2
2025-11-11 15:50:48.493 [info] [startSync]: numJobs: 2
2025-11-11 15:50:48.494 [info] Uploading 2 files.
2025-11-11 15:50:48.494 [info] Total number of files to embed: 146
2025-11-11 15:50:48.494 [info] Not aborted
2025-11-11 15:50:48.494 [info] Starting while loop.
2025-11-11 15:50:48.499 [info] syncing batch of 2 files and 3005 bytes
2025-11-11 15:50:50.778 [info] Completed job successfully: 17505526-9e8b-43db-9335-77c9c68a47af .\JAIMES AF.
Web\Components\Layout\MainLayout.razor
2025-11-11 15:50:50.778 [info] Completed job successfully: 17505526-9e8b-43db-9335-77c9c68a47af .\JAIMES AF.
Web\Components\Pages\NewGame.razor
2025-11-11 15:50:50.821 [info] Finishing sync (hasUpdates: true, isErr: false)
2025-11-11 15:50:51.070 [info] Finished indexing repository.
2025-11-11 15:50:51.070 [info] Indexing job successfully done or aborted.
2025-11-11 15:51:51.268 [error] Error getting submodules: A system error occurred (ENOENT: no such file or directory,
```

Indexing

Indexing & Docs

Indexing & Docs

Codebase

Codebase Indexing ?

Embed codebase for improved contextual understanding and knowledge. Embeddings and metadata are stored in the cloud, but all code is stored locally.

100%

381 files

Sync

Delete Index

Index New Folders

Automatically index any new folders with fewer than 50,000 files



Ignore Files in .cursorignore

Files to exclude from indexing in addition to .gitignore. [View included files.](#)

Edit

Index Repositories for Instant Grep BETA

Automatically index repositories to speed up Grep searches. All data is stored locally.



Docs

Crawl and index custom resources and developer docs

+ Add Doc

- AG-UI Docs

Indexed 12/5/25, 2:17 PM



Tools & MCP Servers

M matt.eland@gmai...
Free Plan

Search settings Ctrl+F

⚙️ General

∞ Agents

→ Tab

📦 Models

☁️ Cloud Agents

🔗 Tools & MCP

📋 Rules, Memories, ...

📖 Indexing & Docs

🌐 Network

>> Beta

📖 Docs 

Tools

Browser



Browser Automation



Ready (Chrome detected)

Installed MCP Servers

No MCP Tools

Add a custom MCP tool here or configure project-specific tools in `<project-root>/cursor/mcp.json`

Add Custom MCP

Tips for Improving Context

Focus

Keep things focused on the task you want the agent to accomplish

New

Start new conversations when ready to move on to new focuses

Filter

Ignore irrelevant file types and directories so they're not indexed

Improve

If you find certain areas are tricky, you may need more rules or tweaks to agent files



Adding Context

Adding Context to our Sample Project

Feedback & Contact Form



<https://forms.gle/SrTzVzFnsDz4kiA18>

Break

Time	Content
8 – 9:15 AM	Introductions, Terminology, and Setup Verification
9:30 – 10:30 AM	Tooling Discussion and Demonstration
10:45 – 11:30 AM	Cursor for Greenfield Development
11:30 AM – 12:30 PM	Lunch Break
12:30 – 1:15 PM	Context Discussion and Sample App Exploration
1:15 – 2 PM	Cursor for Brownfield Development
2:15 – 3:45 PM	Advanced AI Integration
4 – 5 PM	Final discussions and topics of interest

Brownfield Development

Let's see what happens if we just work on production code without any added context



This afternoon's codebase

What do we get out of the box?

Switch to the 'workspace/no-rules' branch and start a new Agent session with the 'step6-historical-feature.md' prompt.

This branch is from a previous point in time for our repo, before the feature described in the prompt was implemented.

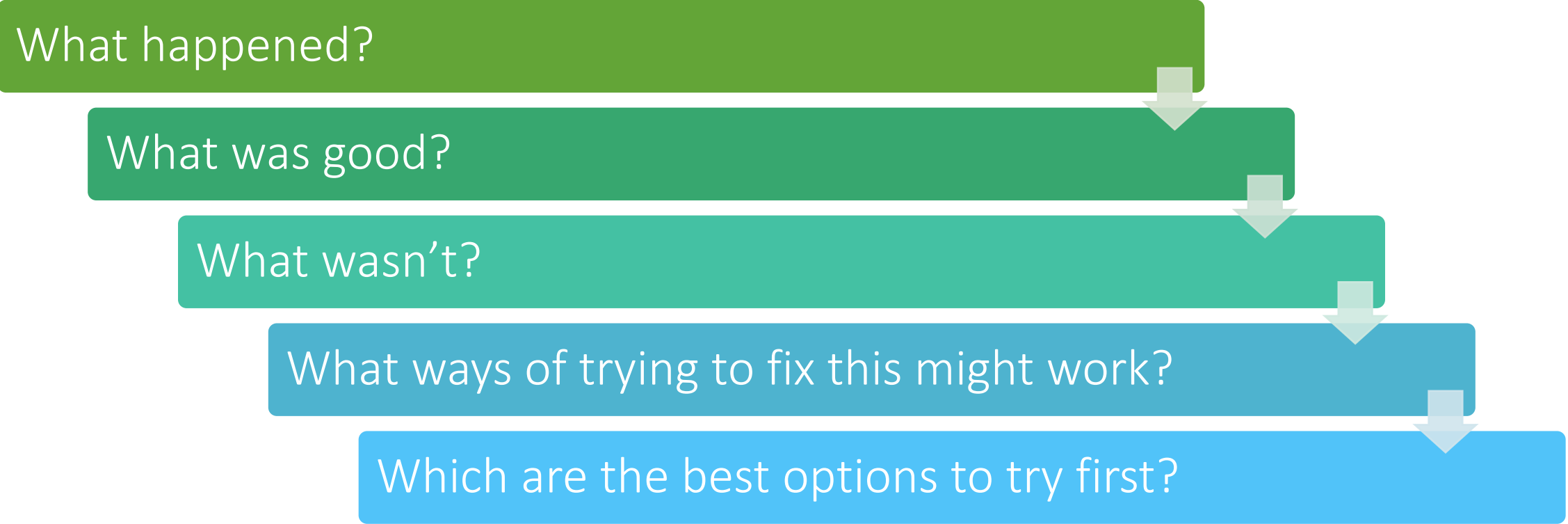
You can either commit changes locally or git stash these results to compare them with an implementation with additional context we'll do later

Demo

Let's try AI assisted development with zero changes!

Discussion

What happened?



```
graph TD; A[What happened?] --> B[What was good?]; B --> C[What wasn't?]; C --> D[What ways of trying to fix this might work?]; D --> E[Which are the best options to try first?];
```

What was good?

What wasn't?

What ways of trying to fix this might work?

Which are the best options to try first?

Feedback & Contact Form



Break

<https://forms.gle/SrTzVzFnsDz4kiA18>

Time	Content
8 – 9:15 AM	Introductions, Terminology, and Setup Verification
9:30 – 10:30 AM	Tooling Discussion and Demonstration
10:45 – 11:30 AM	Cursor for Greenfield Development
11:30 AM – 12:30 PM	Lunch Break
12:30 – 1:15 PM	Context Discussion and Sample App Exploration
1:15 – 2 PM	Cursor for Brownfield Development
2:15 – 3:45 PM	Advanced AI Integration
4 – 5 PM	Final discussions and topics of interest

Advanced AI Integration

Time to add in some contextual sprinkles to see how we can improve things

The background image shows a construction site at sunset. Several tall cranes and scaffolding structures are visible against a sky transitioning from orange to blue. The scene is filled with the silhouettes of construction equipment and buildings under construction.

Project-specific Scaffolding and Prompts

Writing a good AGENTS/Rules.md

Every time you use an Agent, it is seeing your codebase for the first time. This means you should tell it the:

- **What** – is this codebase? What are the features? Where are they defined?
- **Why** – do we have this codebase? What is its purpose? Why are things organized the way they are?
- **How** – do we use/build it? How do you style code? How do you run tests? How do you validate changes?

These rule files are included in *every single chat*, so we want them to be universally applicable to the project. High level, curated rules beats targeted 'hotfix' instructions that might only be applicable in narrow cases.

Bad AGENTS/Rules examples

1. Database Schema Instructions

When creating new database tables:

- Always include `created_at` and `updated_at` timestamps
- Use UUID primary keys, not auto-increment integers
- Add indexes on foreign keys
- Follow naming pattern: `user_profiles`, not `userProfiles`

Why bad: Only relevant when doing database work, wastes context otherwise.

3. One-Time Setup Instructions

Setting up the development environment:

1. Install Node.js 18+
2. Run ``npm install``
3. Copy `.env.example` to `.env`
4. Set up local PostgreSQL
5. Run migrations with ``npm run migrate``

Why bad: Setup happens once, not in every session. Bloats every conversation.

2. Detailed Code Style Guidelines

Code Style:

- Use 2 spaces for indentation
- Max line length 80 characters
- Prefer `const` over `let`
- [50 more formatting rules...]

Why bad: Linters do this better/faster, bloats context.

Teach the LLM to learn from examples.

Better AGENTS/Rules examples

1. Project Structure Overview

Project Structure:

- /apps/web - Next.js customer dashboard
- /apps/api - Go API server
- /apps/admin - Internal admin panel
- /packages/ui - Shared React components
- /packages/db - Database schemas and migrations

This is a pnpm monorepo. Always work from the root directory.

Why good: Helps Claude navigate and understand scope, relevant to all tasks.

3. Technology Stack

Tech Stack:

- Frontend: Next.js 14, TypeScript, TailwindCSS
- Backend: Go 1.21, PostgreSQL 15
- Infrastructure: Docker, deployed on AWS ECS
- Package manager: pnpm (not npm/yarn)

Why good: Informs all technical decisions, always relevant.

2. Progressive Disclosure Pointers

Documentation:

- Database conventions: see `agent_docs/database_design.md`
- API patterns: see `agent_docs/api_guidelines.md`
- Testing approach: see `agent_docs/testing_strategy.md`
- Deployment: see `agent_docs/deployment.md`

Read relevant docs before starting work on those areas.

Why good: Agent only reads what it needs, when it needs it.

Building Rules for your codebase

Create verbose repo overview



Generate rules with supplemented best practices document + artifact



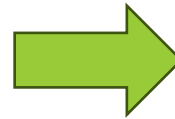
Evaluate rules by re-creating existing features.
Refine as necessary.

Building Rules for your codebase

Start by building a comprehensive architecture document of your codebase. This can be done manually, or by utilizing Agents to break down your codebase piece by piece.

 product-mfe Give me a high-level overview of this repository. I want you to identify the primary components of the system. Determine what it is meant to do, how it does it. Identify common patterns used in the repo. Identify the main services and describe what they do. Create an architecture diagram of the repo.

∞ Agent ▾ gpt-5-codex ▾



Great, now I want you to dig into each one of the components that you identified, and break them out in more granular analysis. Collate this more granular investigation back into the initial high level review you made. Basically dig into the smaller parts and add them back in. Also, be sure to 'escape' mermaid components with parenthesis like this: "Fragments (styles/content/scripts)"

∞ Agent ▾ gpt-5-codex ▾



Ok, I now want you to convert this to a markdown file in the root of the repo, named 'ARCHITECTURE.md'. This should include the diagrams and findings you've made. This should have a consistent structure for all services. Include the mermaid diagrams, but also include a simple ascii tree diagram of the different components and their purpose.

∞ Agent ▾ gpt-5-codex ▾ 1x



Step 0: Switch to 'no-rules' branch

There are 2 prepared branches for this exercise:

- workshop/no-rules
- workshop/with-rules (if you want to skip to the end of the exercise)

workshop/no-rules is a branch made at an older commit in the repo, missing newer features we will use Cursor to re-implement

workshop/with-rules is the same, but with the Rule files we're about to make already prepared

Step 1: Create overview file

This is an example command for starting your ARCHITECTURE.md document. This example is very generic and provides no repo-specific details, but the more direction you can give here, the better.

This command will try to analyze the repo from higher to lower levels, consolidating all the information it finds into a single file named 'ARCHITECTURE.md'

Start a new Agent Session and run the command `/create-overview`.

Learning Rule-Building Principles

LLMs are stateless, they see your codebase for the first time every chat.

We want to give them enough context to start building the way we expect them to, but we don't want them to read the entire codebase every single chat.

- Teach the LLM *how to learn things* about the repo on its own, instead of giving the answer
- Keep Rules concise, reference existing code instead of hard-coding examples.
- Build progressive rules, link to other rules instead of fitting it all in one place.
- [This blog post](#) is a great resource for how to build effective rules.

Step 2: Create Initial Rules

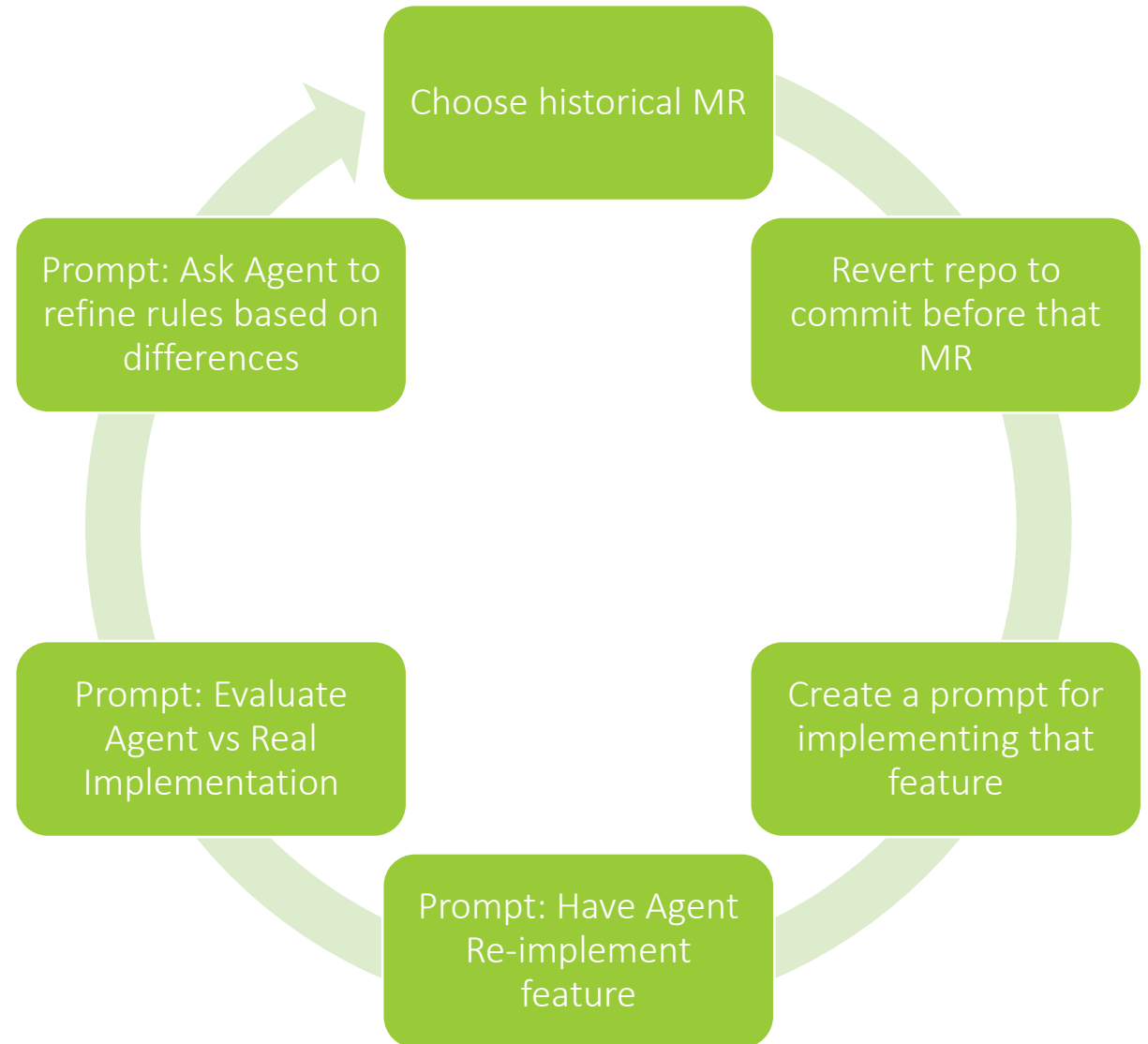
We've got 2 great pieces of context for building Rules, the `ARCHITECTURE.md` for our repo and best practices defined in the `writing-good-rules.md` document.

- Create a new Agent Session and run the command `/create-rules`.
- This will give us the initial set of rules for our codebase, using the initial analysis as a starting point and the best practices document as a guide.
- This is a good starting point, but we will need to review and curate these rules for best results.
- Once you have finished, make a backup of the `.cursor/rules` outside of your repo (easier than dealing with git commands with a whole room of people)

Validate Rules with Known Implementation

How do we know if these rules are good?

Test them out on something we know the 'right' answer to!



Step 3: Let's recreate the feature - with Rules

Now that we know what results look like with no guidance, let's see how our Cursor Rules change the implementation.

Create a new Agent and run the prompt in the file '`historical-feature.md`'.

This version will automatically follow the generated rules built from the previous 'create-rules' command.

Comparing Human vs Agent MRs

Cursor Agents can use git to compare their implementation with the real merge commit.

This gives us a workflow for improving our rules to enforce code that is more similar to historical MRs.

Run prompt '`compare-with-real-feature.md`' in the same session as step 3.

Comparison: My Implementation vs. Commit b497ebd

Files Comparison

File	My Implementation	Real Commit
ANFStoreAttrDeleteService.java	✓ Created	✓ Created
ANFStoreAttrValDeleteService.java	✗ Not created	✓ Created
ANFStoreAttrValRepository.java	✓ Added findByStoreAttr_StoreAttrId()	✗ No changes
Controller	Added 1 endpoint	Added 2 endpoints
Tests	Service + Controller tests	Service + Controller tests

Step 4: Rule Refinement

Now that we have identified the differences between the Agent and Human MRs, we can analyze our rule set to identify what areas we can improve.

Run the prompt in `'analyze-rules-from-implementation.md'` in the same session as before.

This will let the Agent compare its own analysis against the real feature and where the rules came up short for its version of the feature.

Context

Thought for 3s

Checking the SemanticSlicer package namespace and fixing the import:

Searched web dritlich.SemanticSlicer NuGet package namespace C#

Auto-Ran command: dotnet

```
$ dotnet list "JAIMES AF.Workers.DocumentChunking/JAIME:
DocumentChunking.csproj" package
```

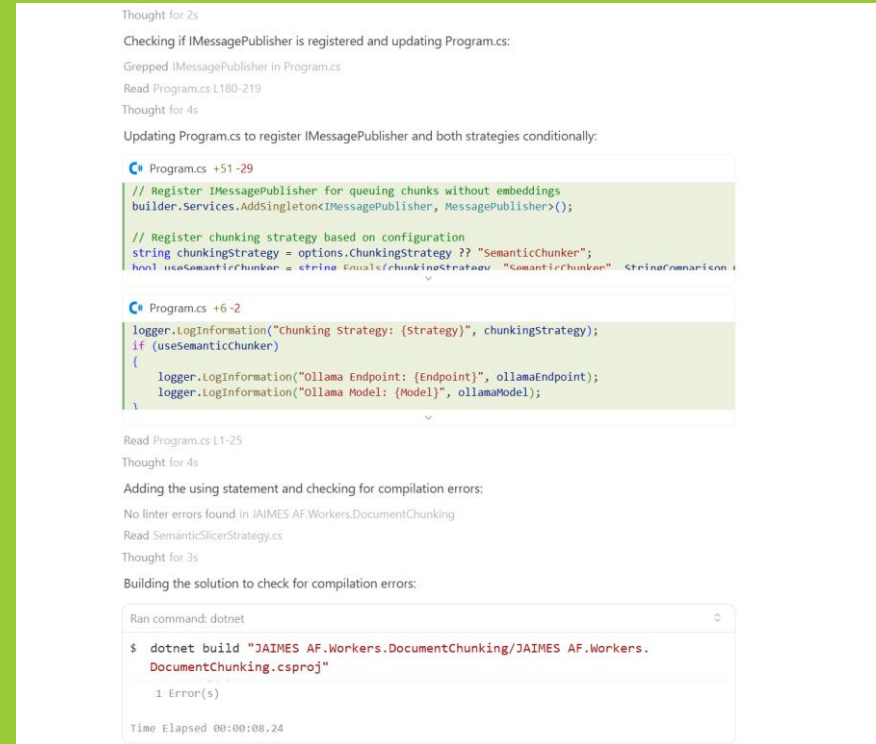
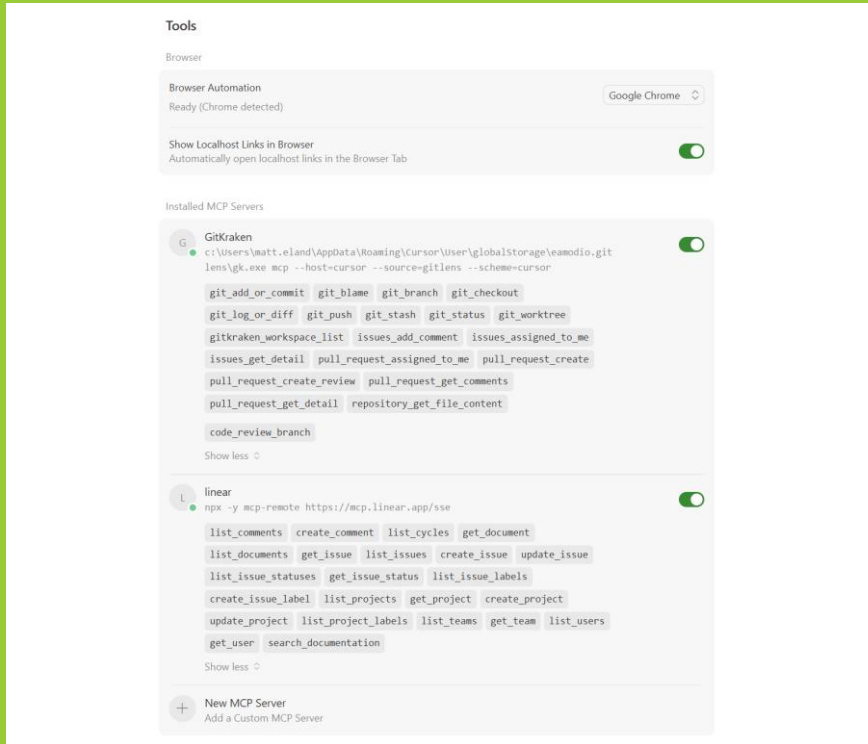
Plan, @ for context, / for commands

∞ Agent ▾ Auto ▾ 1×

38.9% context used

Active Rules
sonarqube_mcp_instructions
testing
AGENTS.md





Tools and Tool Calling

MCP Servers

Installed MCP Servers



GitKraken

c:\Users\matt.eland\AppData\Roaming\Cursor\User\globalStorage\eamodio.gitlens\gk.exe mcp --host=cursor --source=gitlens --scheme=cursor



git_add_or_commit git_blame git_branch git_checkout

git_log_or_diff git_push git_stash git_status git_worktree

gitkraken_workspace_list issues_add_comment issues_assigned_to_me

issues_get_detail pull_request_assigned_to_me pull_request_create

pull_request_create_review pull_request_get_comments

pull_request_get_detail repository_get_file_content

code_review_branch

Show less ↕



linear

npx -y mcp-remote https://mcp.linear.app/sse



Show less



New MCP Server

Add a Custom MCP Server

Browser MCP Support

Tools

Browser

Browser Automation

Ready (Chrome detected)

Google Chrome 

Show Localhost Links in Browser

Automatically open localhost links in the Browser Tab



Featured MCPs

Browse MCPs or [post a MCP](#) to reach 250,000+ monthly active developers.

Add new + ← →

**Atono**

Atono's MCP server connects your AI coding assistant directly to your product backlog, pulling your...

**Endgame**

Deployment superpowers for Cursor. Fast + Self-healing + Free - endgame.dev...

**Mailtrap Email Sending**

Integrates with Mailtrap Email Platform to enable sending transactional emails via API.

**MailerSend MCP**

Access the transactional messaging MailerSend API and services directly from your AI coding environment or...

Search 1800+ MCPs

**Ashra MCP Server**

Extract structured data from any website with a simple SDK call. No scraping code, no...

**Compresto MCP**

A Model Context Protocol (MCP) server for Compresto, providing AI assistants with...

**Caiyun Weather**

Caiyun Weather is a weather data service that provides real-time weather...

**Encore**

Encore turns Cursor into more than a coding tool. The open source framework lets you...

**urlDNA**

Scan and analyze potentially malicious URLs using the urlDNA

**Pearl**

The Pearl MCP Server expands the ecosystem of MCP-compatible tools by...

**AWS Knowledge MCP**

A fully managed remote MCP server that provides up-to-date documentation, code...

**AWS API MCP Server**

The AWS API MCP Server enables AI assistants to interact with AWS services...

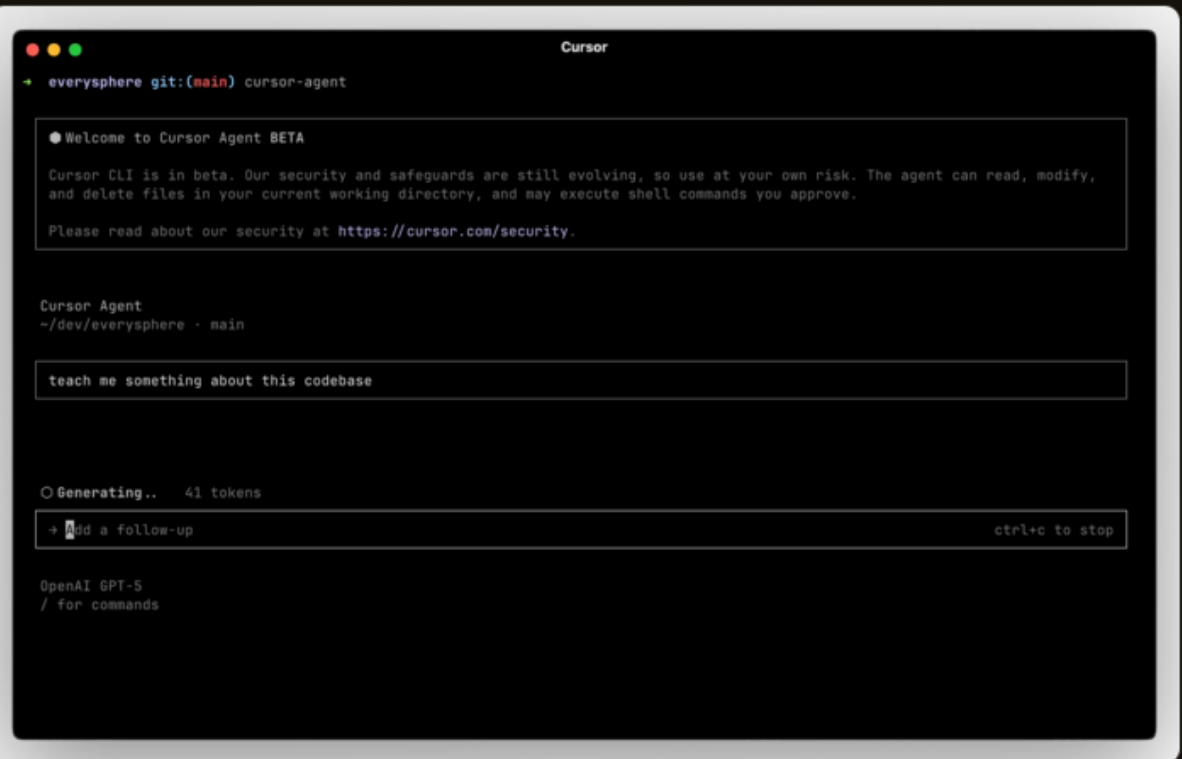
External MCP Servers

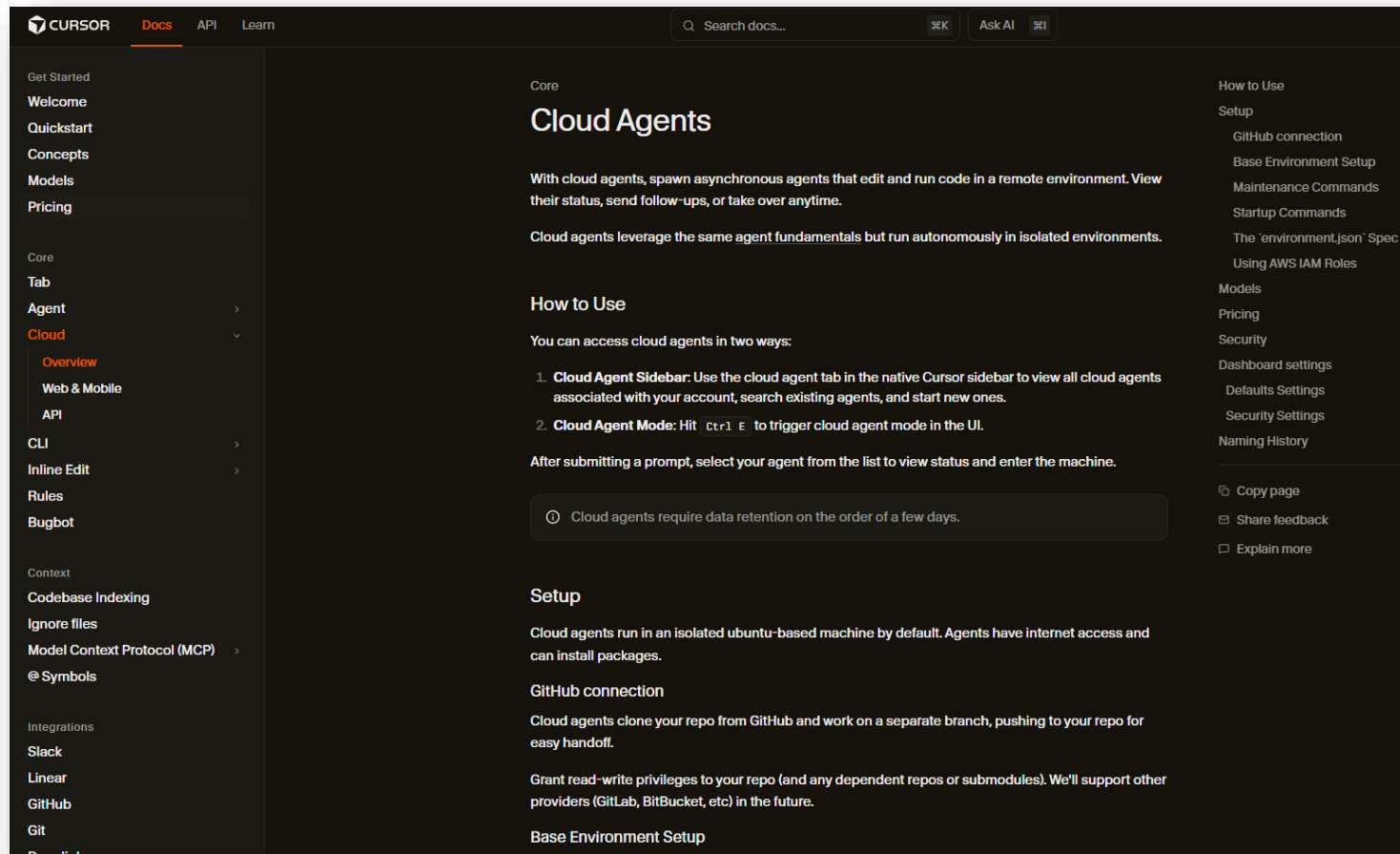
<https://cursor.directory/mcp>

Cursor CLI

Getting started

```
1 # Install
2 curl https://cursor.com/install -fsS | bash
3
4 # Run interactive session
5 cursor-agent
```





Cloud Agents

Bugbot

 **Cursor** bot reviewed 2 days ago

backend/server/src/bugbotGithub/applyBugbotPRReview.ts

```
228 - const reviewCommentsResponse = await octokit.request('GET /repos/{owner},
228 + const reviewCommentsResponse = await octokit.request('GET /repos/{owner},
229     owner: repository.owner,
230     repo: repository.repo,
231     pull_number: pullRequest.issueNumber,
232 - review_id: commentResponse.data.id,
232 + since: new Date(Date.now() - 10000).toISOString(), // Only get recent co
233 });
```

 **Cursor** bot 2 days ago

Bug: PR Comments Fetching Misses New Inline Comments (Potential edge case)
By switching to all PR comments and relying on a fixed 10s since filter without pagination, Bugbot can miss new inline comments — either due to timing delays or because they aren't included in the first batch of results.

 [Fix in Cursor](#) [Fix in Web](#)

Bugbot Add-on

Free

\$0

Includes:

- ✓ Limited code reviews each month
- ✓ Unlimited access to Cursor Ask
- ✓ Cursor connection to auto-fix bugs
- ✓ GitHub integration

Try Bugbot

Pro

\$32/user/mo.

Everything in Free, plus:

- ✓ 14 day individual trial
- ✓ Unlimited reviews on up to 200 PRs/month
- ✓ Access to Bugbot Rules

Get Bugbot Pro

Teams

\$32/user/mo.

Everything in Pro, plus:

- ✓ 14-day team trial
- ✓ Unlimited code reviews on all PRs
- ✓ Analytics and reporting dashboard
- ✓ Advanced rules and settings

Get Bugbot Teams

Enterprise

Custom

Everything in Teams, plus:

- ✓ 30-day org-wide trial
- ✓ Advanced analytics and reporting
- ✓ Priority support and account management

Contact Sales

Bugbot Editions

Hands-on Exercise

Add targeted context, then repeat some of our steps from earlier

Discussion

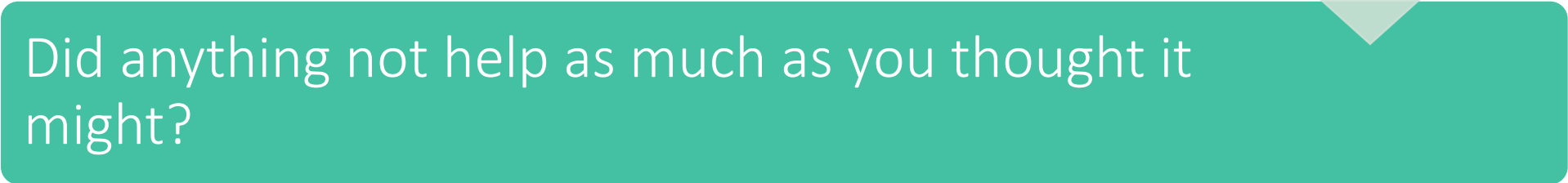
How did this go?



What was most effective?



Did anything not help as much as you thought it might?



Feedback & Contact Form



Break

<https://forms.gle/SrTzVzFnsDz4kiA18>

Time	Content
8 – 9:15 AM	Introductions, Terminology, and Setup Verification
9:30 – 10:30 AM	Tooling Discussion and Demonstration
10:45 – 11:30 AM	Cursor for Greenfield Development
11:30 AM – 12:30 PM	Lunch Break
12:30 – 1:15 PM	Context Discussion and Sample App Exploration
1:15 – 2 PM	Cursor for Brownfield Development
2:15 – 3:45 PM	Advanced AI Integration
4 – 5 PM	Final discussions and topics of interest

Demo

Generating and Improving Test Coverage

Bonus Content?

As time allows

+

•

○

Conclusion

Final thoughts, recommendations, and next steps / resources

Recommendations for Long-Term Success

1

Store context in project-specific rules

2

Bootstrap your initial context, then review it

3

Start in a narrowly scoped manner and expand

4

Update documentation and context as things change

5

Get Cursor's help flagging weak areas

Injecting Context in Cursor

Conversational
prompt

Code and
Comments

Agents.md

Rules

Memories

Commands

Team-specific
Context

User-specific
Context

Indexed
Documents

External tools /
MCP Servers

Adopting AI Tooling

01

Start with a specific scope in mind

- Team / Project
- Codebase
- Tooling

02

Pick work items that are well-defined, speced out, and deal with common technologies

03

Plan in advance for testing and code review

04

Provide relevant context and instructions for how to operate (tests, docs, building code, etc.)

05

Consider pre-built Agent files as a starting point

Onboarding new developers with AI in mind



AI USE IS EXPECTED,
BUT NOT A SUBSTITUTE
FOR COMMON SENSE



TESTING IS CRITICAL



CODE REVIEWS BECOME
EVEN MORE IMPORTANT



THE WHOLE TEAM
ALWAYS OWNS
SOFTWARE QUALITY



DON'T TRUST THE AI

Additional Resources

<https://blog.LearningEDJE.com>

<https://cursor.com/docs>

<https://cursor.directory/learn>

<https://Agents.md>

Open Q&A



Feedback & Contact Form



<https://forms.gle/SrTzVzFnsDz4kiA18>